TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY



Final Year Project Report on

"कानुन" - APPLICATION FOR PERFORMING RAG OPERATION ON CONSTITUTION OF NEPAL

For the partial fulfillment of the requirements of the degree of
Bachelor of Science in Computer Science and Information Technology (B.Sc.CSIT)
awarded by Tribhuvan University

Submitted By:

Abhinab Shrestha (T.U. Exam Roll No. 25818/77)
Manish Kumar Rajbanshi (T.U. Exam Roll No. 27251/077)

ST. XAVIER'S COLLEGE

Maitighar, Kathmandu, Nepal

Submitted To:

Office of the Dean

Institute of Science and Technology

Tribhuvan University

Kathmandu

Feb, 2025



SUPERVISOR RECOMMENDATION

This is to certify that the final year project entitled **"कानुन" - APPLICATION FOR PERFORMING RAG OPERATION ON CONSTITUTION OF NEPAL** is an academic work completed by **Abhinab Shrestha (T.U. Exam Roll No. 25818/77)** and **Manish Kumar Rajbanshi (T.U. Exam Roll No. 27251/077)** submitted in the partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology awarded by Institute of Science and Technology, Tribhuvan University under my guidance and supervision.

The information presented by them in the project report has not been submitted earlier to the best of my knowledge.

……..

Er. Bishnu KC
Lecturer
St. Xavier's College
Maitighar, Kathmandu



ST. XAVIER'S COLLEGE

DEDICATED TO EXCELLENCE, LEADERSHIP, SERVICE

A HIGHER EDUCATION INSTITUTE RUN BY THE NEPAL JESUIT SOCIETY

| MAITIGHAR, KATHMANDU, NEPAL | www.sxc.edu.np |

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Department of Computer Science, St. Xavier's College for acceptance, a Final Year Project Report entitled **"कानुन" - APPLICATION FOR PERFORMING RAG OPERATION ON CONSTITUTION OF NEPAL** submitted by **Abhinab Shrestha (T.U. Exam Roll No. 25818/77)** and **Manish Kumar Rajbanshi (T.U. Exam Roll No. 27251/077)** for the partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Information Technology awarded by Tribhuvan University.

……..

Er. Bishnu KC
Project Supervisor
Lecturer
St. Xavier's College

……..

Er. Rajan Karmacharya
Internal Examiner
Chief Technology Officer
St. Xavier's College

……

Mr. Ganesh Yogi
HoD, Department of Computer Science
St. Xavier's College

……..

External Examin

ACKNOWLEDGEMENT

First and foremost, we are most grateful to be students of Computer Science here in St. Xavier's college, utterly packed by expertise of the respective field, greatly supportive to the lemurs. We would like to express our sincere gratitude to **Er. Bishnu KC,** our supervisor for creating a virtuous academic and social environment to foster this project. We would like to express our innermost thanks to him for providing us with all the crucial advice, guidance and resources for the accomplishment of this project.

We would also like to express our gratitude to **Mr. Ganesh Yogi** - Head of Department for his constant support and encouragement throughout the completion of this project.

We are grateful to the entire Computer science department of St. Xavier's College for their constant support and guidance. At the same time, we would like to express our sincere thanks to all our friends and others who helped us directly or indirectly during this project work.

ABSTRACT

The Constitution is the foundation of a nation's legal system, justice, equality, and the protection of citizens' rights. Despite its significance, the complexity of constitutional language often creates a knowledge gap, leaving citizens, legal professionals, and government officials struggling to access and comprehend key provisions.

"कानुन" (Kanun) is an AI Chat Application built on a OpenAI model. This innovative platform is designed to understand Nepal's Constitution which provides accurate, accessible, and contextually relevant information. Through advanced Natural Language Processing (NLP) techniques, " कानुन " bridges the gap between legal language and its users, empowering them with knowledge about their constitutional rights and responsibilities.

The application employs a user-friendly interface and robust query-processing capabilities to deliver precise answers to user inquiries. By leveraging research in AI and legal informatics, the system is tailored to address the challenges of interpreting legal texts while maintaining high levels of accuracy and reliability.

This report details the problem statement, objectives, and methodology behind the development of " कानुन " It also explores the potential of LangChain to transform legal services by democratizing access to constitutional knowledge, thereby fostering legal literacy and adherence to constitutional principles.

Keywords: LangChain, Constitution, Legal Literacy, Natural Language Processing (NLP), AI Chat Application, Nepal

TABLE OF CONTENTS

ACKNOWLEDGEMENT i

ABSTRACT ii

TABLE OF CONTENTS iii

LIST OF ABBREVIATIONS v

LIST OF FIGURES vi

LIST OF TABLES vii

CHAPTER 1: INTRODUCTION 1

1.1 INTRODUCTION 1

1.2 PROBLEM STATEMENT 1

1.3 OBJECTIVE 2

1.4 SCOPE AND LIMITATIONS 2

1.5 DEVELOPMENT METHODOLOGY 3

1.5.1 PROJECT FRAMEWORK 3

1.5.2 IMPLEMENTATION: 4

1.6 RERPORT ORGANIZATION 5

CHAPTER 2: LITERATURE REVIEW 7

2.1 BACKGROUND STUDY 7

2.1.1 CHALLENGES IN ACCESSING LEGAL INFORMATION 7

2.1.2 RETRIEVAL AUGMENTED GENERATION 7

2.1.3 VECTOR DATABASE 8

2.1.4 VECTOR SEARCH AND SEMANTIC RETRICAL IN LEGAL AI 8

2.1.5 RAG FOR CONSTITUTIONAL ASSISTANTS 8

2.2 LITERATURE REVIEW 9

CHAPTER 3: SYSTEM ANALYSIS 16

3.1 SYSTEM ANALYSIS 16

3.1.1 REQUIREMENT ANALYSIS 17

3.1.2 FEASIBILITY STUDY 26

3.1.3 ANALYSIS 28

CHAPTER 4: SYSTEM DESIGN 34

4.1 SYSTEM DESIGN 34

4.1.1 COMPONENT DIAGRAM 34

4.1.2 DEPLOYMENT DIAGRAM 34
4.2 ALGORITHM DETAILS 35
4.2.1 MAXIMUM MARGINAL RELEVANCE 35
4.2.2 COSINE SIMILARITY 35
4.2.3 RETRIEVAL AUGMENTED GENERATION 37
4.2.4 TRANSFORMER ARCHITECTURE 38
CHAPTER 5: IMPLEMENTATION AND TESTING 42
5.1 IMPLEMENTATION 42
5.1.1 TOOLS USED 42
5.1.2 IMPLEMENTATION DETAILS OF MODULES 46
5.2 TESTING 54
5.2.1 UNIT TESTING 55
5.2.2 SYSTEM TESTING 59
5.3 RESULT ANALYSIS 60
5.3.1 PROPOSED MODEL 60
5.3.2 ADVANTAGES OF PROPOSED MODEL 61
5.3.3 DISADVANTAGES OF PROPOSED MODEL 61
5.3.4 RELEVANT SCREENSHOT 62
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION 68
6.1 CONCLUSION 68
6.2 FUTURE RECOMMENDATION 68
REFERENCES 70

LIST OF ABBREVIATIONS

API	Application Programming Interface
LLM	Large Language Model
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
DBMS	Database Management System
IG	Information Gain
JSON	JavaScript Object Notation
SQLite3	Standard Query Language

LIST OF FIGURES

Figure 1: Incremental Development 3
Figure 2: Project Development Methodology 5
Figure 3: Use Case Diagram for Normal User 19
Figure 4: Use Case Diagram for Staff Management 20
Figure 5: Use Case Diagram For Admin Control 20
Figure 6: Gantt Chart of system development schedule 28
Figure 7: Object Modeling using Class Diagram for Kanun 29
Figure 8: Dynamic modeling of Normal User Authentication Flow using sequence diagram 30
Figure 9: Dynamic modeling of Staff User Workflow using sequence diagram 31
Figure 10: Dynamic modeling of Super User Workflow using sequence diagram 32
Figure 11: Process modeling of RAG-Based Application 33
Figure 12: Object Modeling using Class Diagram of RAG- Based Application 34
Figure 13: Transformer's Architecture 38
Figure 14: Cover Page 62
Figure 15: Cover Page - Introduction 62
Figure 16: Cover Page - Contact 63
Figure 17: Login Page 63
Figure 18: Registration Page 64
Figure 19: Dashboard Page 64
Figure 20: User Management 65
Figure 21: Chat Log 65
Figure 22: Injestion Page 66
Figure 23: Conversation Page 66
Figure 24: Chat Page 67
Figure 25: News Page 67

LIST OF TABLES

Table 1: Use Case Login Table 21
Table 2: Use Case Register Table 22
Table 3: Use Case Dashboard Table 22
Table 4: Use Case Generate Text/Image Table 23
Table 5: Use Case Convert PDF to JSON Table 24
Table 6: Use Case User Management Table 24
Table 7: Use Case Logout Table 25
Table 8: Time Schedule 28
Table 9: Test Case For User Registration 55
Table 10: Test Case For User Login 56
Table 11: Test Case For Dashboard 56
Table 12: Test Case For Chat Module 57
Table 13: Test Case For Chat Module 58
Table 14: System Testing 59

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

The Constitution is the fundamental law of a country. Any law inconsistent with this Constitution shall, to the extent of such inconsistency, be void[1]. Understanding the Constitution is essential for citizens, legal professionals, and government officials as it helps protect democracy and ensures adherence to rights and responsibilities. However, the complexity of legal language often creates a knowledge gap, making it difficult for the general population to access and interpret constitutional information.

"कानुन" (KANUN) is a Retrieval-Augmented Generation (RAG)-based system designed to simplify access to constitutional information with precision and reliability. By utilizing the capabilities of Large Language Models (LLMs) alongside an advanced retrieval framework, "कानुन" ensures that the information provided is not only relevant but also rooted in authentic sources, such as the Constitution of Nepal. Unlike conventional AI models that rely primarily on pre-existing training data, कानुन actively searches for and extracts relevant constitutional references before formulating responses. This dynamic approach enhances accuracy, keeps legal interpretations aligned with official documents, and provides users with precise insights into constitutional provisions.

By merging retrieval-based methodologies with sophisticated language models, कानुन delivers a well-structured and dependable solution for navigating constitutional law. Its ability to provide direct and authoritative references makes it a crucial tool for legal experts, academics, and anyone seeking clarity on Nepal's constitutional framework.

1.2 PROBLEM STATEMENT

The Constitution act is the backbone of the legal system serving the rights of the citizens, ensuring justice and responsibilities. However, accessing and understanding constitution acts is quite challenging due to its complexity structure of legal documents and which can lead to a knowledge gap where citizens are not fully aware of their constitutional acts rights and responsibilities. In the absence of an accessible and interactive tool to address

constitution acts related queries where citizens will be unaware about their constitutional rights and the legal action.

The problem is made worse by the lack of simple, accessible tools that provide clear, accurate information on constitutional matters. Using advanced AI and natural language processing, " कानुन " will make it easy for users to access and understand constitutional laws and related legal acts. Whether for ordinary citizens or government officials, this application will serve as a vital tool to help navigate the complexities of the law, increase legal awareness, and ensure compliance with constitutional rights and duties.

1.3 OBJECTIVE

The major objectives of the "कानुन" (KANUN) are listed below:

1. To parse the constitutional PDF data into JSON format consisting of parts, articles, clauses, sub-clauses, explanation by matching regex pattern.
2. To provide a secured and user friendly interface for prompting queries regarding the constitution of Nepal.
3. To provide an information interface for understanding the functioning of LLM operation related to RAG operation happening in real time.

1.4 SCOPE AND LIMITATIONS

The project aims to make Nepal's constitution more accessible and user-friendly by converting it into a structured format that allows users to easily search and retrieve relevant information. It provides a smart system where people can ask questions related to constitutional laws and receive meaningful answers. This helps in simplifying complex legal information, making it easier for citizens, students, and researchers to understand their rights and responsibilities. The system is designed to assist in legal education and awareness, ensuring that people can quickly find accurate information without having to go through lengthy documents. Additionally, it has the potential to expand by including other legal texts, making it a valuable resource for anyone seeking legal knowledge.

While the project makes legal information more accessible, it has certain limitations. The accuracy of responses depends on how well the information is extracted and structured. Since legal language can be complex, the system may sometimes misinterpret queries or

provide incomplete answers. Additionally, it does not replace professional legal advice and should only be used as a reference. The system may also require regular updates to reflect changes in laws or amendments to the constitution. Furthermore, it relies on sufficient computing resources to function efficiently, which could be a challenge for large-scale usage.

1.5 DEVELOPMENT METHODOLOGY

1.5.1 PROJECT FRAMEWORK

A project management framework is a set of processes, tasks and tools that provide guidance and structure for the execution of a project. It helps to map out the progression of the individual project steps, from beginning to completion. The framework includes all aspects of the project, from required resources and tools to specific processes and tasks.

This system uses an Incremental development model. Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed [2]

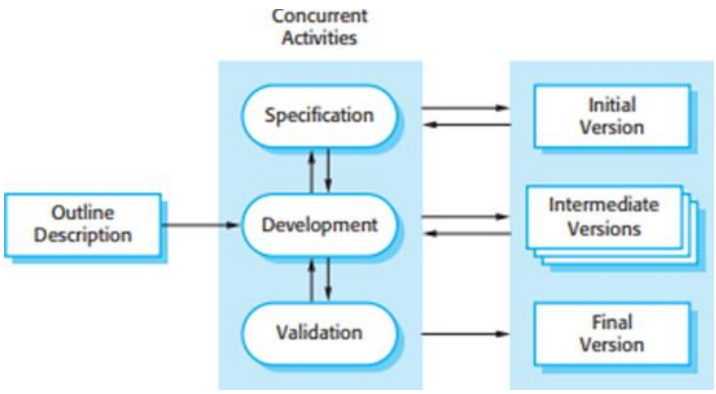


Figure 1: Incremental Development

1.5.1.1 REQUIREMENT ANALYSIS:

This is the first phase of the incremental model, the developer identified the requirements. And the system functional requirements are understood. To develop the software under the incremental model, this phase performs a crucial role

1.5.1.2 DESIGN AND DEVELOPMENT:

In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

1.5.1.3 TESTING:

In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

1.5.2 IMPLEMENTATION:

Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

The process begins when the user logs into the system, ensuring authentication and access control. Once logged in, the system checks whether an existing conversation is available. If not, a new one is initiated; otherwise, the user continues from where they left off. The user then inputs a query related to Nepal's constitution, which is processed by the system. The input message is converted into a vector format using an embedding model to capture its semantic meaning.

Next, the system compares the embedded query with pre-stored constitutional data using cosine similarity to determine the closest match. The most relevant legal text is then retrieved from a vector database. This retrieved content is processed to generate a meaningful and structured response based on the legal information found.

Finally, the system displays the generated response to the user, ensuring clarity and accuracy. The user can either continue asking further questions or end the conversation. This approach ensures that responses are contextually relevant, precise, and efficient in retrieving information while maintaining a seamless interaction.

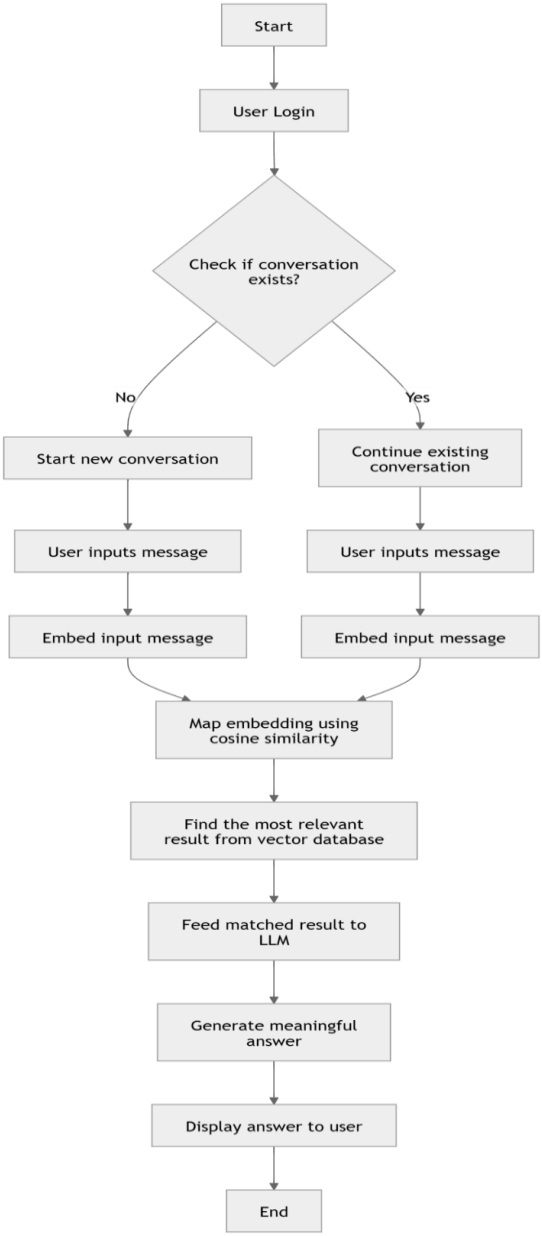


Figure 2: Project Development Methodology

1.6 RERPORT ORGANIZATION

This report meticulously details the journey of developing a system for the intricate task of categorizing Constitutional Laws. The project's narrative unfolds across five distinct chapters, each dedicated to a crucial stage in the system's creation. Initially, the report lays the groundwork with a comprehensive introduction, articulating the project's central purpose, the specific problem it seeks to resolve, and the overarching objectives that guided its development. This initial chapter also carefully delineates the project's scope,

acknowledges any inherent limitations, and meticulously outlines the methodological approach employed throughout the entire process.

Following this introduction, the report delves into the theoretical underpinnings of the project with a thorough background study. This section explores the fundamental theories, core concepts, and specialized terminology relevant to the complex field of Constitutional Law categorization. Complementing this theoretical exploration, a comprehensive review of existing literature is presented, examining prior research, related projects, and scholarly contributions in this domain. This review provides crucial context and highlights the project's place within the broader landscape of legal informatics.

The report then transitions to the critical phase of system analysis. This chapter meticulously details the process of eliciting and defining the system's requirements. A thorough requirements analysis is conducted, identifying both the functional needs the system must fulfill and the non-functional constraints it must adhere to. This analysis is followed by a rigorous feasibility study, evaluating the project's viability from multiple perspectives: technical, operational, economic, and scheduling. The analysis phase culminates in a structured or object-oriented representation of the system, employing appropriate diagrams and models to capture its essential structure and behavior.

Building upon the foundation laid in the analysis chapter, the report proceeds to describe the system's design. This section elaborates on the chosen architectural approach, detailing the rationale behind it and providing detailed specifications for the various components and modules that comprise the system. The design chapter also includes a discussion of the algorithms employed within the system, explaining their logic and purpose in the context of Constitutional Law categorization.

The report then documents the practical realization of the system in the implementation and testing chapter. This section outlines the specific tools and technologies used in the development process, explaining their roles and contributions. It describes the implementation of key modules, providing insights into the technical details of the system's construction. Furthermore, this chapter presents the test cases employed to validate the system's functionality and performance, along with a detailed analysis of the test results.

Finally, the report concludes with a summary of the project's achievements and a reflection on the lessons learned. This concluding chapter also looks forward, discussing potential future enhancements and directions for continued development of the system. It offers recommendations for extending the system's capabilities and adapting it to evolving needs in the field of Constitutional Law.

CHAPTER 2: LITERATURE REVIEW

2.1 BACKGROUND STUDY

Access to legal information, particularly constitutional law, has traditionally been challenging due to its complex language and structure. Individuals often rely on legal professionals or extensive document searches to find relevant legal information. However, Natural Language Processing (NLP) advancements, particularly Retrieval-Augmented Generation (RAG), have enabled AI-powered legal assistants to provide more efficient and context-aware responses to constitutional queries [3]

2.1.1 CHALLENGES IN ACCESSING LEGAL INFORMATION

Legal documents contain technical language, making it difficult for non-experts to understand them. Furthermore, conventional search engines that operate using keyword-based search often return irrelevant or incomplete results if the exact wording differs from the search query. In Nepal, language barriers and the lack of structured digital legal databases further hinder access to legal knowledge. Research has shown that AI-driven legal assistants can bridge this gap by offering automated and accessible legal support [4].

2.1.2 RETRIEVAL AUGMENTED GENERATION

The RAG (Retrieval-Augmented Generation) model is designed for text generation tasks, performing a process that involves retrieving information from given source data and utilizing that information to generate desired text. RAG model consists of the data processing pipeline for RAG usage, involving breaking down the original data into smaller chunks and converting text data into numerical vectors through embedding, which are then stored in a vector repository[5]

2.1.3 VECTOR DATABASE

A vector database is a novel type of database developed to address the problem of long-term memory deficiency in LLMs. This database is specialized in efficiently storing and managing high-dimensional real-valued vector indices. Unlike traditional databases, a vector database represents queries in the form of real-valued vectors (embeddings) and supports a method to extract data with similar vectors. Therefore, it is optimized for storing and utilizing vectors obtained as results from AI models. [5]

2.1.4 VECTOR SEARCH AND SEMANTIC RETRICAL IN LEGAL AI

Traditional search methods rely on exact word matching, which is inefficient for legal research. Vector embedding, which represent text as numerical vectors, have transformed legal information retrieval by enabling semantic similarity-based searches. By applying cosine similarity, legal queries can be matched with relevant constitutional texts regardless of variations in wording [4]. Vector databases, such as FAISS and Pinecone, are widely used to store and retrieve vectored legal documents, significantly enhancing search accuracy and efficiency [6].

2.1.5 RAG FOR CONSTITUTIONAL ASSISTANTS

The RAG framework integrates retrieval-based search with AI-powered response generation. Unlike traditional virtual assistants that rely on predefined responses, RAG-based systems dynamically retrieve the most relevant constitutional provisions and generate structured, legally accurate responses. This approach ensures that users receive fact-based, context-aware answers instead of purely generated text without legal grounding [6].

2.1.6 LLM TRAINING STAGES

LLM training stages consists of following stages: First of all data is gathered(Data Corpus), data extraction, splitting chunks, embedding, vector store, prompt & search & generation

In Nepal, the need for such a solution is significant considering the complexities of the legal system. The purpose of LangChain based application "कानुन" aims to address this necessity by leveraging cutting-edge AI technology and natural language processing. "कानुन" intends to be a dependable and user-friendly platform for answering constitutional related questions, warning citizens, legal experts and government representatives alike.

2.2 LITERATURE REVIEW

The constitution is the most important document of the country which sets the framework for the governing of the country. It serves as a guide to ensure justice, equality and the protection of citizen's rights. The Constitution is generally written in complex language. In order to interpret the laws lately various LangChain related researches have been carried out due to its ability to process and understand the natural language.

Oguzhan Topsakal, and Tahir Cetin Akinci paper titled "Creating large language model applications utilizing LangChain:: A primer on developing LLMs apps fast." focuses on the utilization of LLMs for the development of applications and their capabilities in a range of tasks, with OpenAI's ChatGPT being popular among millions of users. The methods and materials in this paper are to enable developers to conveniently utilize other data sources and interact with other applications. Large Language Models (LLMs) are used as a type of model in LangChain to accept a text string (prompt) and output a text string. As a stateless model, each transaction to the LLMs API endpoint is independent. Vector stores and retrievers can take multiple forms as memory insertion. The use case of common end-to-end applications includes autonomous agents, chatbots, code understanding agents, extraction question answering over documents, and analyzing structured data [8].

Jeong, Cheonsu paper titled "A study on the implementation of generative ai services using an enterprise data-based LLMs application architecture." focuses on the recent advancements in generative AI technology this paper addresses the challenge of information scarcity and proposes remedies by harnessing the LLMs capabilities. The methods in this paper liberal framework for implementing generative AI service by effectively combining and orchestrating various technologies. The implementation procedure of the RAG model by source data collection and extraction, chunk generation, embedding, building the vector database, integration of prompt and search results. The author classifies the LLM and RAG models on reducing the size and complexity of LLM models to achieve effective results with smaller and simpler models. The combined efforts of these initiatives are expected to expand the utilization of generative AI technology which will be practical in the real world.[9]

In their 2024 study titled “A Large Language Model Agent-Based Legal Assistant for Governance Applications,” Mamalis, Kalampokis, Fitsilis, Theodorakopoulos, and Tarabanis explored the role of large language models (LLMs) in providing substantial assistance to various stakeholders, including decision-makers, administrators, and citizens. This research paper focused on the design and implementation of an LLM-based legal assistant tailored for interacting with legal resources. The working mechanisms of such an LLM can be observed in the regulatory activities of major industrial powers. An unexpected outcome of scaling LLMs has been a qualitative shift in their behavior, affecting their capabilities. Furthermore, developing and testing the LLM-based system with complex legal questions remains a challenge. The authors highlight the future potential of these systems in effectively performing legal tasks [10].

In the study conducted by Filipovska, E., Mladenovska, A., Bajrami, M., Dobreva, J., Hillman, V., Lameski, P. and Zdravevski, E. in 2024, September, the focus was on Benchmarking OpenAI’s APIs and other Large Language Models for Repeatable and Efficient Question Answering Across Multiple Documents. The researchers implemented this to evaluate the efficiency and repeatability of OpenAI’s APIs and other LLMs in automating question-answering tasks across multiple documents, specifically focusing on analyzing data privacy policy documents. The limitation or research gaps in the research manuscript is focusing on the foremost constraint affecting the response quality is the prompt format- during the execution of the model whether it didn't meet the satisfactory criteria. The efficiency of LLM can be improved with a strategic selection of the embeddings and vector base. The authors also conclude that using OpenAI models via API calls is a useful workaround for speeding up response times in the event that improving local infrastructure is not practical for lengthy texts. The main target is not to use human experts to simply validate AI output, but rather to use AI output to make the human work effectively and efficiently.[11]

In their research title “L.E.G.A.L.(Leveraging Expert Guidance for AI in Law): A RAG-Based System for Legal Document Navigation”, Hui Chia, Geordie Z. Zhang, Daniel Russo-Batterham, Kabir Manandhar Shrestha, Uy Thinh Quang, Zeyu Wang, Jeannie Marie Paterson address the two critical challenges in the legal domain that has the opacity of AI-driven tools, often referred to as “black boxes” and the risk of generating hallucinated content that is not grounded in reality. The method used in the research paper is to build a

RAG framework designed to assist users in querying legal documents and receiving reliable answers. A legal dataset curated with expert-verified questions, answers and supporting references to benchmark the system.The contribution by the authors in this paper is to host the system for public access to primary users which are the law students and legal professionals. [12]

In their 2024 study titled “Revolutionizing Retrieval-Augmented Generation with Enhanced PDF Structure Recognition”, Demiao LIN address the RAG as predominant method in the field of professional knowledge-based question answering where model companies embedding and chat api interfaces and frameworks like LangChain have already integrated the RAG process. The limitation or research gaps in the research manuscript is focusing on ranking and token limit issue and fine segmentation drawback which led to retrieving only part of the required information but not the key content within the table content. The author classifies the experiments on the impact of PDF Recognition on RAG through settings, data preparation and results of extractive question on case studies of RAG. More compared to deep learning-based document parsing methods to give a more liberal understanding of the relationship between the RAG quality and document parsing quality.[13]

In their 2013 study titled “The Great Disruption: How Machine Intelligence Will Transform the Role of Lawyers in the Delivery of Legal Services”, McGinnis, John O., and Russell G. Pearce address the great disruption faced by the legal profession. It shows great impact on traditional journalism for decreasing the employment of about 17,000 people in the last eight years and the market value of newspapers. The common objections that lawyers have always adapted to technological changes and these changes will create legal action about their proper scope and demand for legal services. The author classifies how regulatory barriers will not prevent machine intelligence from undermining lawyers' monopoly and at the same time legal profession has continued to promote unauthorized practice of law status which create barriers to machine intelligence for providing legal service to non-lawyers offering legal services. Ultimately, the disruptive effect of machine intelligence will trigger the end of lawyer monopoly and benefit to society and clients as legal services become more transparent and affordable to consumers.[14]

In their research title “IQLS: Framework for leveraging Metadata to enable Large Language Model based queries to complex, versatile Data”, Azirar Sami, Hossam A. Gabbar, and Chaouki Regoui mainly focus on optimizing models to fulfill this task on given required data in a structured format. Retrieving, consolidating and applying the right data is therefore a huge task. This paper presents the query complexity where the IQLS system is designed to handle various complex and diverse queries with ease. Its ability to handle language ambiguity is one of its key features which enhances user experience by accurately. The IQLS extends the capabilities of LLMs to more complex tasks like route planning while still making use of the available data structures. The adaptability of the models strongly depends on the data. It maps the user query to the data based on the schema which influences the results to the wording of the query. This study contributes the practical testing in real use environment on the model, future research could identify more error-proof ways to parse complex structured outputs from LLMs to enable more versatile applications.[15]

In their research title “Rethinking legal judgement prediction in a realistic scenario in the era of large language models”, Shubham Kumar Nigam, AniketDeroy2 Subhankar Maity, Arnab Bhattacharya address legal judgment prediction systems are to predict and explain judgment as cases arrives on the bench for adjudication and to enhance both the prediction and explanation tasks by grounding principles that are applied in real cases. The methodology used in this study to extract relevant sentences from legal judgments using Hierarchical BiLSTM-CRF classifiers on different rhetorical roles. Let combined input and response length for these models is 4096 to kens, the inputs into chunks if 2048 words and segmentation will allow us to generate judgment predictions as one token corresponds to three-quarters of a word which translates to about 750 words for 1000 tokens. The limitation or research gaps in the research manuscript is focused solely on Supreme Court of India (SCI) judgments, which may limit the generalizability of the models to other courts or jurisdictions. The author explored the effectiveness of various LLMs and transformer architectures in the task of judgment prediction and explanation using the ILDC-multi dataset. [16]

In their research title “LegalReasoner: A Multi-Stage Framework for Legal Judgment Prediction via Large Language Models and Knowledge Integration”, Wang Xuran, Xinguang Zhang, Vanessa Hoo, Zhouhang Shao and Xuguang Zhang address the

framework demonstrates substantial improvements over state-of-the-art baselines, achieving an average accuracy increase of 7.8% across all datasets. Legal Judgment Prediction (LJP) system main target is to forecast the outcomes of legal cases based on the facts presented, relevant statutes and historical precedents. The task is filled with challenges that push the boundaries of current artificial intelligence capabilities. The methodology in this paper present a detailed description of our LegalReasoner framework as legal knowledge infusion, case-law retrieval, multi-hop reasoning, judgement synthesis, training and optimization. The author addresses the complex challenges of legal reasoning by seamlessly combining several innovative components. This framework improves AI-assisted legal decision-making, increasing efficiency and consistency in legal processes. [17]

In their 2019 study titled "Artificial intelligence and machine learning based legal application: the state-of-the-art and future research trends.", Riya Sil, Abhishek Roy, Bharat Bhushan, and A. K. Mazumdar addresses that AI and machine learning have the potential to augment human capabilities in the legal field. AI and machine learning have been increasingly used in the legal field and are expected to have a significant impact on legal practices in the future. This paper provides an overview of the current state and future research trends of artificial intelligence (AI) and machine learning application in the legal field. The methodology in this paper analyses legal information based on semantics and makes legal predictions from the legal data set and helps the judiciary system in automation which increases the efficiency within an affordable budget. The paper gives the information that in the next twenty years more than 1,00,000 jobs have been automated in the field of law. Its estimation proves that automation is 22% of a lawyer's job and 35% of a clerk's job can be done. The professionals should embrace the transformation with the adoption of AI conventional system.[18]

In their research title "(A) I Am Not a Lawyer, But...: Engaging Legal Experts towards Responsible LLM Policies for Legal Advice.", Inyoung Cheong, King Xia, K.J. Kevin Feng, Quan Ze Chen and Amy X. Zhang addresses when and why LLMs should or should not provide legal advice and presents a framework for responsible LLM policies in this domain. The study presents a 4-dimension framework for determining the appropriateness of LLM responses in legal contexts. The experts recommended LLM systems focus on helping users identify relevant information rather than providing definitive legal

judgments. The case-based deliberation method used can be applied to engage expert perspectives on LLM response appropriateness in other professional domains beyond law. The limitation or research gaps in the research manuscript is focusing on their experience with current LLM technology and their evaluations will change as the technology evolves. It didn't involve end-users of legal services and future research was investigated end-user perceptions and compare them to the expert-informed results. [19]

In their 2024 study titled "It cannot be right if it was written by AI: on lawyers' preferences of documents perceived as authored by an LLM vs a human.", Jakub Harasta, Tereza Novotná and Jaromir Savelka addresses that lawyers and students perceive legal documents they believe were generated by an Al language model versus those believe were crafted by a human. The methods and materials in this paper was to recruit a sample of 75 lawyers and student participants with legal documents that were explained as if it was either human-crafted or AI-generated. To evaluate the documents on their correctness and language quality. The author's major findings addresses that lawyers and law students have a clear preference for legal documents where participants believe that written by humans over those they believe were generated by AI. This could be helpful for guiding the responsible adoption of AI-generated legal documents and inform discussion about how legal processes should develop to accommodate these technology changes. The limitation or research gaps in the research manuscript is focusing on the actual legal consequences of using AI-generated documents only the perceptions of lawyers and law students and didn't directly address how legal processes should be updated to accommodate AI-generated documents and further research and discussion is needed in this area.[20]

In their research titled "LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain.", Nicholas Pipitone and Ghita Houir Alami address the benchmark designed to evaluate the retrieval step of RAG pipelines within the legal space. It serves as a critical tool for companies and researchers focused on enhancing the accuracy and performance of RAG systems in the legal domain. In this paper methodology is used to extract highly relevant text snippets from legal documents rather than retrieving entire documents and retracing the context used in LegalBench queries back to their original location in a legal corpus. The limitation or research gaps in the research manuscript is focusing on the four sources of dataset include NDAs, M&A agreements, various commercial contracts and privacy policies of consumer-facing online companies. The

creation of both LegalBench-RAG and its lightweight counterpart and LegalBench-RAG-mini addresses a critical gap in the evaluation of RAG system particularly in contexts where legal accuracy is paramount.[21]

In their 2024 study titled "Leveraging open-source models for legal language modeling and analysis: a case study on the Indian constitution.", Vikhyath Gupta addresses about legal language modeling and analysis using open-source models from hugging face which holds promise for efficient processing and analysis of legal texts. The methods and materials in this paper are to enable the preprocessing of the Constitution data and split it into chunks and also use ChromaDB and LangChain Vector Stores to store and manage the data. The approach of the author not only streamlines the process of model creation but also ensures that the resulting LLMs are tailored to the unique characteristics of legal texts within a specific jurisdiction or legal framework Their privacy protection such as data access control, minimal data exposure and limited dependencies. These case study highlight on the Indian Constitution exemplifies the transformative impact on enabling legal professionals and policymakers to navigate extensive legal text with ease.[22]

Although chatbots have existed for decades, the emergence of transformer-based large language models (LLMs) has captivated the world through the most recent wave of artificial intelligence chatbots, including ChatGPT. Transformers are a type of neural network architecture that enables better contextual understanding of language and efficient training on massive amounts of unlabeled data, such as unstructured text from the internet. As LLMs have increased in size, their improved performance and emergent abilities have revolutionized natural language processing. Since language is integral to human thought, applications based on LLMs have transformative potential in many industries. In fact, LLM-based chatbots have demonstrated human-level performance on many professional benchmarks, including in radiology. LLMs offer numerous clinical and research applications in radiology, several of which have been explored in the literature with encouraging results. Multimodal LLMs can simultaneously interpret text and images to generate reports, closely mimicking current diagnostic pathways in radiology. Thus, from requisition to report, LLMs have the opportunity to positively impact nearly every step of the radiology journey. Yet, these impressive models are not without limitations. This article reviews the limitations of LLMs and mitigation strategies, as well as potential uses of

LLMs, including multimodal models. Also reviewed are existing LLM-based applications that can enhance efficiency in supervised settings.

In conclusion, LLM integration into the legal domain holds great promise for democratizing the access to legal knowledge all over the glove. However, the complexity and interpretation ability might be the factor to choose the correct requirements.

CHAPTER 3: SYSTEM ANALYSIS

3.1 SYSTEM ANALYSIS

System analysis is a problem-solving method that involves examining the broader system, deconstructing its components, and understanding how it operates to achieve specific goals. The first step in addressing a system-related problem is to analyze the system itself [23]. This process includes breaking down the system into its constituent parts and evaluating how these parts function collectively. In some cases, understanding a system may involve disabling certain components to observe the impact or modifying parts to analyze the resulting changes.

System analysis is conducted to study a system or its components to identify its objectives. It enhances system performance and ensures that all components operate efficiently to fulfill their intended purposes [24]. It also involves studying business processes, gathering operational data, understanding the flow of information, identifying bottlenecks, and developing solutions to overcome system weaknesses, thereby supporting the achievement of organizational goals.

Furthermore, system analysis includes subdividing complex processes involving the entire system, identifying data stores, and analyzing manual processes. According to the *Merriam-Webster Dictionary*, system analysis is defined as “the process of studying a procedure or business in order to identify its goals and purposes and create systems and procedures that will achieve them in an efficient way” [25].

3.1.1 REQUIREMENT ANALYSIS

Requirement analysis is a critical phase in system development that involves identifying, documenting, and analyzing the needs and expectations of stakeholders to ensure the system meets its intended objectives. It focuses on understanding what the system should do, the constraints it must operate within, and how it will interact with other systems [26].Requirements analysis involves various tasks that help engineers understand stakeholder demands and explain them in simple and visual ways. It is essential to a software or system project's success.

For a project to be successful, its requirements must be testable, actionable, documented, measurable, traceable. Requirements analysis involves various stakeholders, such as project sponsors, throughout the project as well as end users whose inputs are most important. The best results typically occur when all parties work together to develop a high-quality requirements document .

There are many requirements of the system which should be analyzed so that effective output can be produced with available resources. Following requirement analysis is carried out for the effectiveness of our system.

i. FUNCTIONAL REQUIREMENTS

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks. So, it’s important to make them clear both for the development team and the stakeholders.[27]

Functional Requirements serve as a comprehensive and detailed set of specifications, outlining the specific tasks and functionalities that a software system or its components must perform. These specifications delineate the system's behavior, covering aspects such as input processing, output generation, and the underlying processes required for specific functions. They encompass diverse areas, including data manipulation, calculations, user interactions, and business procedures. The primary objective of these requirements is to guide the design and development of the software, ensuring it aligns with the intended users' needs .

Functional requirement gathers and analyzes the functional part of the system. In 'The LangChain Based Chat Application System', users are allowed to view laws and regulations or sign up and log in the dashboard. The users are allowed to search for constitutional queries. The users are allowed to give the text as an input to the system. Once the input is passed, the given text is then processed through various NLP techniques. The system analyzes the input into relevant articles and clauses from the Constitution of Nepal. Functional requirement of the system can be described with the help of use case diagram as following:

Use Case Diagram

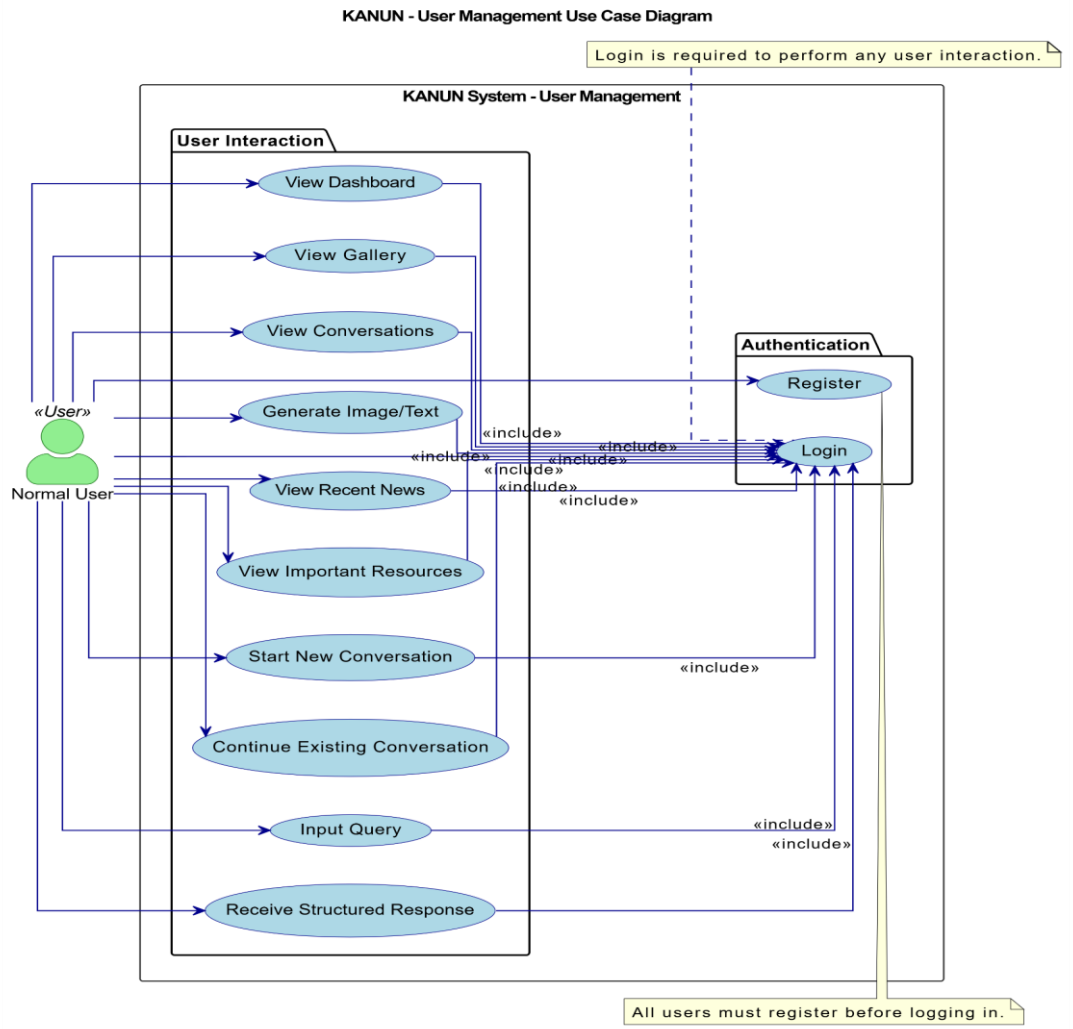


Figure 3: Use Case Diagram for Normal User

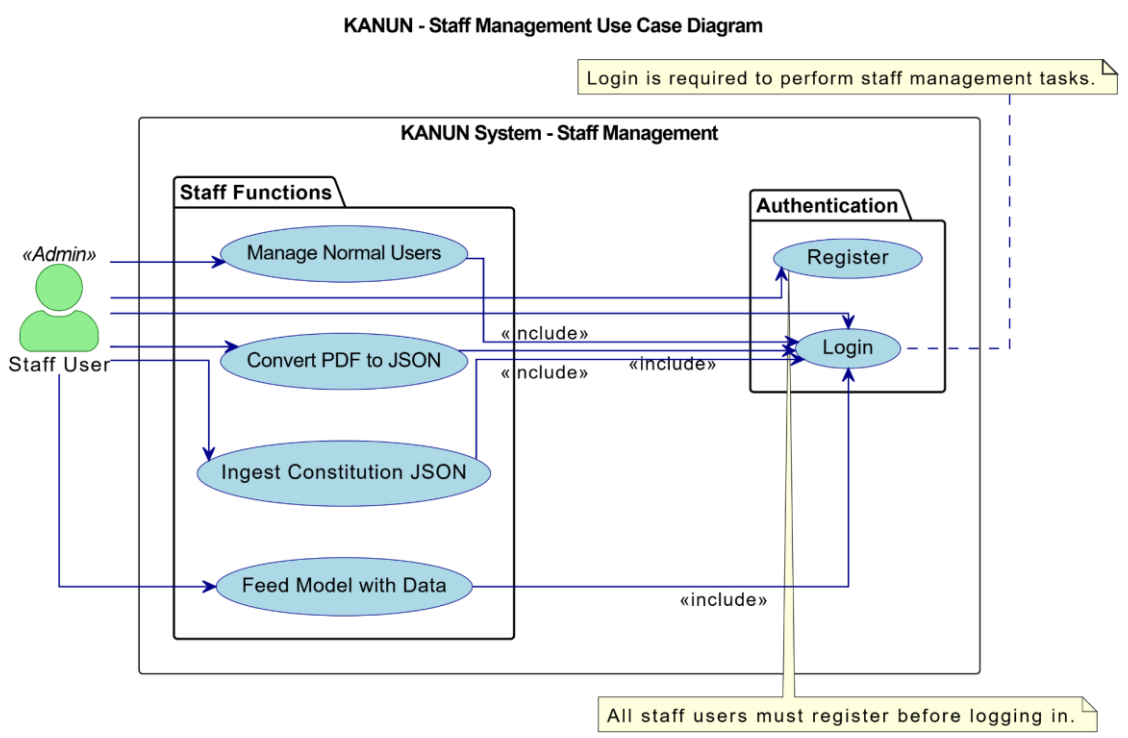


Figure 4: Use Case Diagram for Staff Management

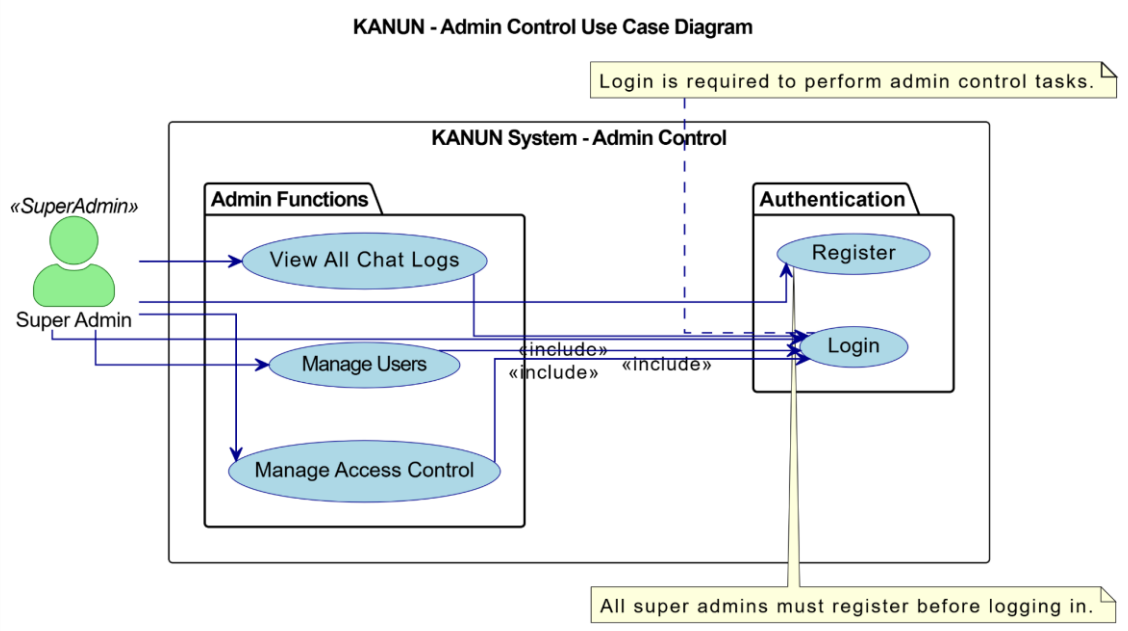


Figure 5: Use Case Diagram For Admin Control

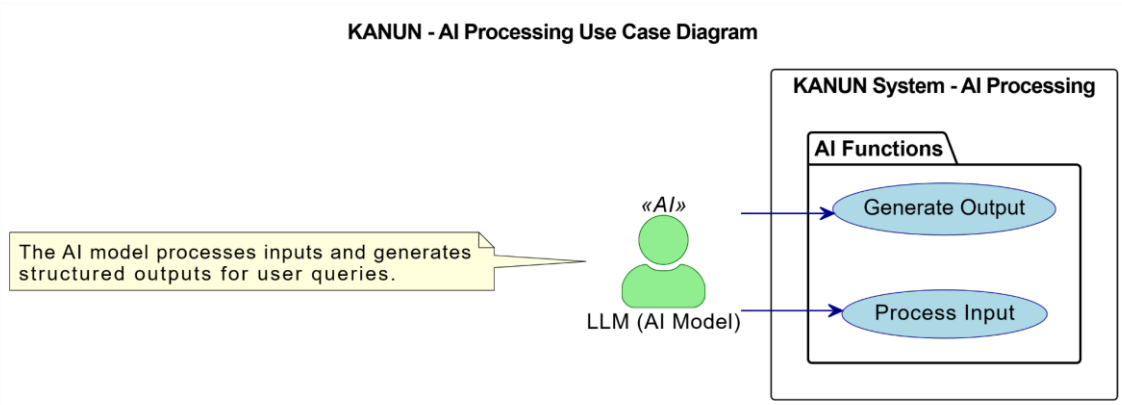


Figure 2: Use Case diagram for AI Processing

USECASE TABLE

Table 1: Use Case Login Table

Fields	Details
Use Case	Login
Actor(s)	Normal User, Staff User, Admin
Description	This use case allows users to log in to the system using their credentials
Priority	High
Status	Implemented
Pre-Condition	The user must have valid login credentials stored in the system
Post-Condition	The user is authenticated and redirected to their respective dashboard
Main Flow	The user enters their credentials, submits the login form, and is authenticated by the system
Alternate Flow	If the user enters incorrect credentials, an error message is displayed, and they can retry

Exceptional Flow	If the system encounters an error (e.g- server downtime), an error message is displayed.

Table 2: Use Case Register Table

Fields	**Details**
Use Case	Register
Actor(s)	Normal User
Description	This use case allows users to register for an account in the system.
Priority	Medium
Status	Implemented
Pre-Condition	The system must allow registration and the user must provide valid details(e.g- email, password).
Post-Condition	The new user account is created and stored in the system and users can login.
Main Flow	The user fills out the registration form with required details and submits it. The system validates the input and creates the account.
Alternate Flow	If the user provides incomplete or invalid details, the system displays an error message.
Exceptional Flow	If account creation fails due to a system error, the user is prompted to retry later.

Table 3: Use Case Dashboard Table

Fields	**Details**
Use Case	Dashboard
Actor(s)	Normal User, Staff User
Description	This use case allows users to view their personal dashboard which includes token usage, recent news, quiz game and resources.
Priority	Medium

Status	Implemented
Pre-Condition	The user must have register credentials with valid user id and password.
Post-Condition	The user can perform necessary action in the system after successful login.
Main Flow	The user navigates to the dashboard and the system displays the dashboard features and content.
Alternate Flow	If no data is available then the system displays a default message.
Exceptional Flow	If the dashboard fails to load then the login failed message has to be displayed.

Table 4: Use Case Generate Text/Image Table

Fields	**Details**
Use Case	Generate Text/Image
Actor(s)	Normal User
Description	This use case allows users to generate text or images using the LLM.
Priority	Medium
Status	Implemented
Pre-Condition	The user must be logged in.
Post-Condition	The generated text/image is displayed to the users.
Main Flow	The user inputs a prompt,submits it and the LLM generates the output.
Alternate Flow	If the user input is invalid then the system displays an error message.
Exceptional Flow	If generation fails then the system logs the error and displays a message.

Table 5: Use Case Convert PDF to JSON Table

Fields	Details
Use Case	Convert PDF to JSON
Actor(s)	Staff User
Description	This use case allows staff users to convert PDF files into JSON format for ingestion.
Priority	Medium
Status	Implemented
Pre-Condition	The PDF files must be uploaded to the system.
Post-Condition	The PDF is converted into JSON format and stored into the system.
Main Flow	The staff user uploads the PDF and the system convert it into JSON.
Alternate Flow	If the PDF is invalid, the system displays an error message.
Exceptional Flow	If conversion fails, the system logs the error and displays a message.

Table 6: Use Case User Management Table

Fields	Details
Use Case	User Management
Actor(s)	Admin
Description	This use case allows super users to manage user accounts, including adding, editing and deleting users and assigning roles.
Priority	High
Status	Implemented
Pre-Condition	The actor must be logged in with appropriate permissions.

Post-Condition	User accounts are updated, added or removed from the system.
Main Flow	The actor navigates to 'User Management', performs an action and confirms the update.
Alternate Flow	If the action is canceled, no changes are made.
Exceptional Flow	If an account management fails, the system logs the error and displays a message.

Table 7: Use Case Logout Table

Fields	**Details**
Use Case	Logout
Actor(s)	Normal User, Staff User, Admin
Description	This use case allows users to log out of the system.
Priority	High
Status	Implemented
Pre-Condition	The user must be logged in.
Post-Condition	The user session is terminated and they are redirected to the login page.
Main Flow	The user clicks the logout button and the system terminates their session.
Alternate Flow	If the logout action is canceled then the session remains active.
Exceptional Flow	If logout fails due to a system error, the session remains active and the system logs the error.

ii.NON FUNCTIONAL REQUIREMENTS

Nonfunctional Requirements are intended to specify system qualities, various system attributes that are not directly related to their functionality. These attributes do not tell what the system does but how well it does it .[28]

The non-functional requirements of this RAG-based application are as follows:

a) **Performance**: Since backend handles all the complex tasks, users will be able to access the query application easily and fast with minimum hardware and average internet connection using the frontend.
b) **Look and feel**: The user interface is easy to use and light colors are used with beautiful design to enhance usability.
c) **Security**: User has to login with valid credentials to access advanced functionalities. Their credentials are stored securely into a database using robust hashing techniques to ensure data privacy and security.
d) **Reliability**: Once constitutional articles and clauses are stored in the database for quick retrieval. So, every time a user met with accurate and relevant information without delays making the system highly dependable.

3.1.2 FEASIBILITY STUDY

A feasibility study is simply an assessment of the practicality of a proposed project plan or method. This is done by analyzing technical, economic, legal, operational and time feasibility factors. The study is also intended to identify possible challenges and problems that we may encounter while completing the project.[29]

Feasibility study helped the authors to determine the strength and weakness of our system. The contents and recommendation of this feasibility study helped us a sound basis for determining how to proceed further. It also helped in making decisions regarding which software to use, hardware combination etc. Some feasibility studies carried out are as follows.

3.1.4.1 TECHNICAL FEASIBILTY

Technically, the system is a RAG-application which is developed to solve Constitutional Queries using LLM, RAG, vector stores and Django Python as core programming language. The backend will use SQLite3 for database management. The frontend is written using HTML, CSS and Bootstrap. The technologies that are required for the development of the system are readily available.

3.1.4.2 OPERATIONAL FEASIBILITY

The "कानुन" (KANUN) application's operational feasibility rests on several interconnected pillars. Robust infrastructure, encompassing appropriate servers, compatible software

(including the Django framework, a language model, and potentially a vector database), and reliable internet connectivity, forms the foundation. Human resources are equally critical, demanding skilled technical personnel alongside legal experts for content accuracy. Operational processes, including data updates, system maintenance, and user support, must be well-defined and consistently executed. Security and privacy, addressed through robust measures and regulatory compliance, are non-negotiable. Finally, scalability and a sustainable funding model are essential for long-term viability. A comprehensive feasibility assessment, covering resources, costs, and risks, is crucial for a successful implementation. Ongoing monitoring and evaluation will be key to optimizing operations and ensuring the application's lasting effectiveness.

3.1.4.3 ECONOMIC FEASIBILITY

The economic feasibility of "कानुन" considers the cost associated with developing, implementing, and maintaining the RAG-based application, as well as the potential returns and savings it can generate. Preprocessing and classifying textual documents for performing RAG operations on the Constitution of Nepal is a task that modern hardware can efficiently handle. The system is lightweight and doesn't require much bandwidth so low-cost servers can also be used. So, this system is economically feasible.

3.1.3.4 SCHEDULE FEASIBILITY

This system has been developed to partially fulfill requirements of BSc. CSIT course. We had 3-4 months to complete the system. The lifecycle of the system can be executed as shown in the schedule below.

Table 8: Time Schedule

Name	Begin Date	End Date	Duration (Days)
Brainstorming	09/25/2024	10/02/2024	8
Project Planning	10/03/2024	10/12/2024	10
Requirement Analysis	10/13/2024	10/19/2024	7
Collecting Resources	10/20/2024	10/28/2024	9
Developing the System	10/29/2024	01/15/2025	79
Testing	01/16/2025	02/10/2025	26
Documentation	09/25/2024	02/28/2025	157

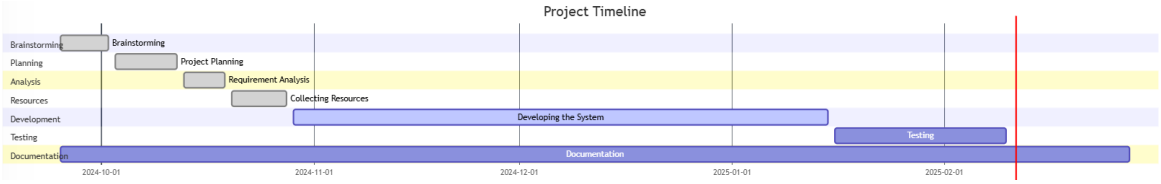


Figure 6: Gantt Chart of system development schedule

3.1.3 ANALYSIS

3.1.3.1 OBJECT MODELING

The object Model sees an information system as a set of objects and classes. The reader will get an in-depth understanding of the object model and its elements. The basic factors of an object model are classes and objects. An object is a physical component in the object-oriented domain which may be tangible such as a person, car or intangible such as a project. A class is a representation of objects, It represents a group of objects that have similar properties and exhibit an expected behavior. [30]

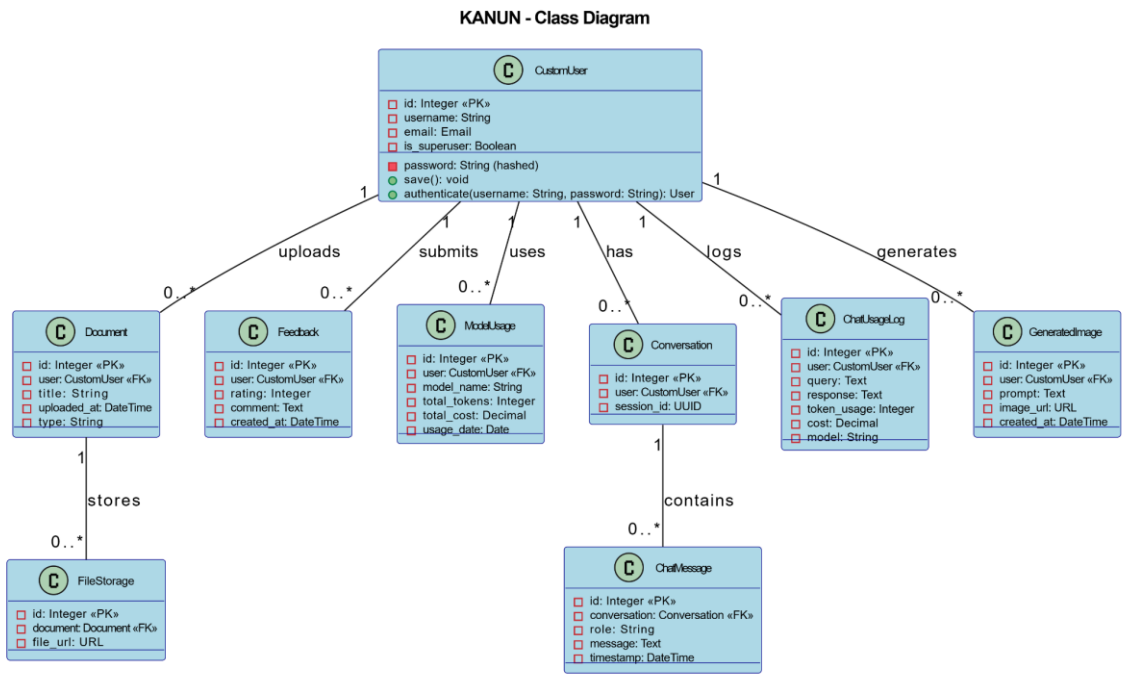


Figure 7: Object Modeling using Class Diagram for Kanun

3.1.3.2 DYNAMIC MODELING

System diagrams are powerful tools that help you to understand how complex systems work. They're particularly helpful for showing you how a change in one factor may impact elsewhere. They're excellent tools for flushing out the long-term impacts of a change. Importantly, a good systems diagram will show how changing a factor may affect itself. Drawing a systems diagram is a good way of starting to build a computer model. The technique helps you to map out the structure of the system to be modeled. It shows the factors and relationships that are important, and helps you to start quantifying the linkages between factors The sequence diagram below shows the sequence in which objects in the system operate with one another.

A normal user visits the dashboard by entering a query or interacting with the system. The query is sent to the backend which is processed through LangChain and RAG pipeline. Relevant articles and clauses are fetched from the Constitution Database and contextual data is retrieved from the Vector Store. The processed response is then displayed to the users. Users can also upload raw PDF files containing constitutional text which are ingested and processed into the Vector Store for future queries or conversation.[31]

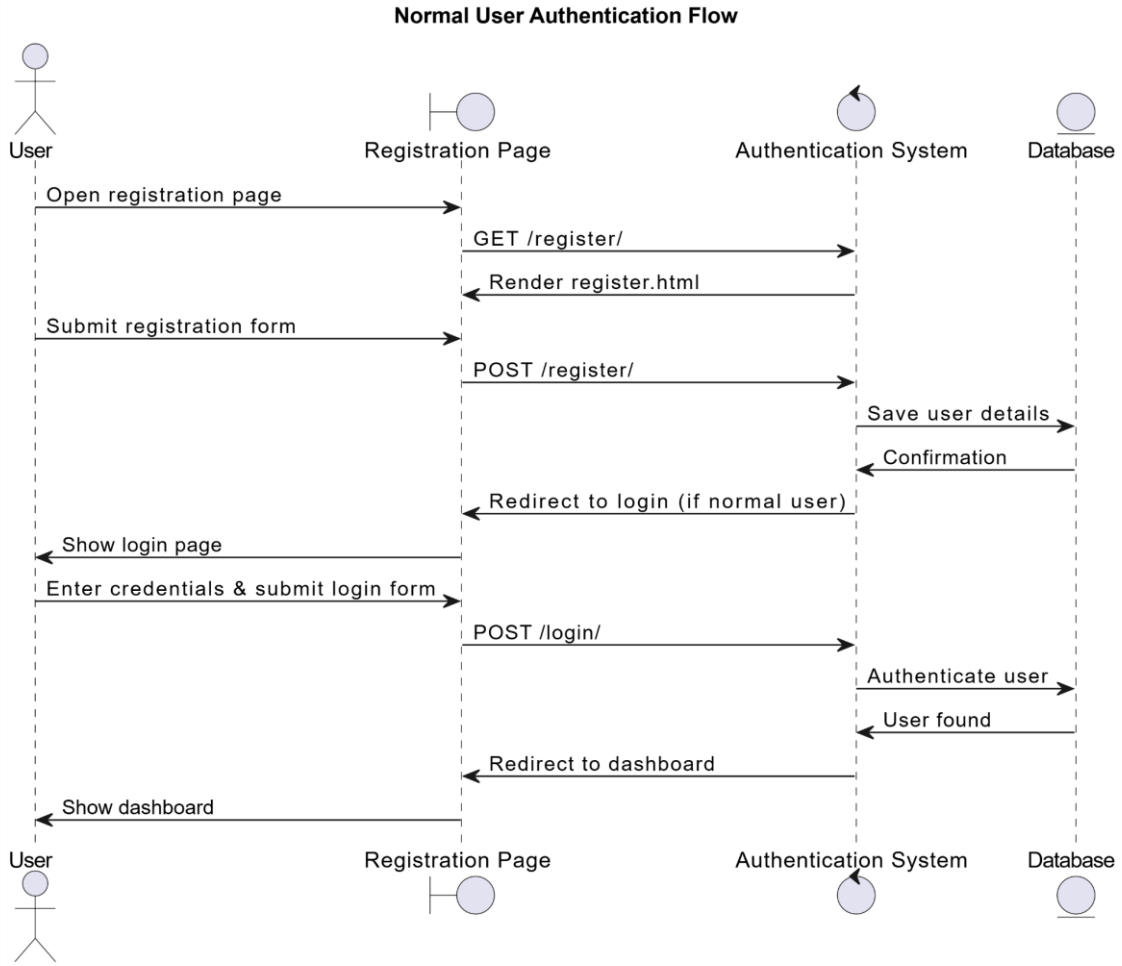


Figure 8: Dynamic modeling of Normal User Authentication Flow using sequence diagram

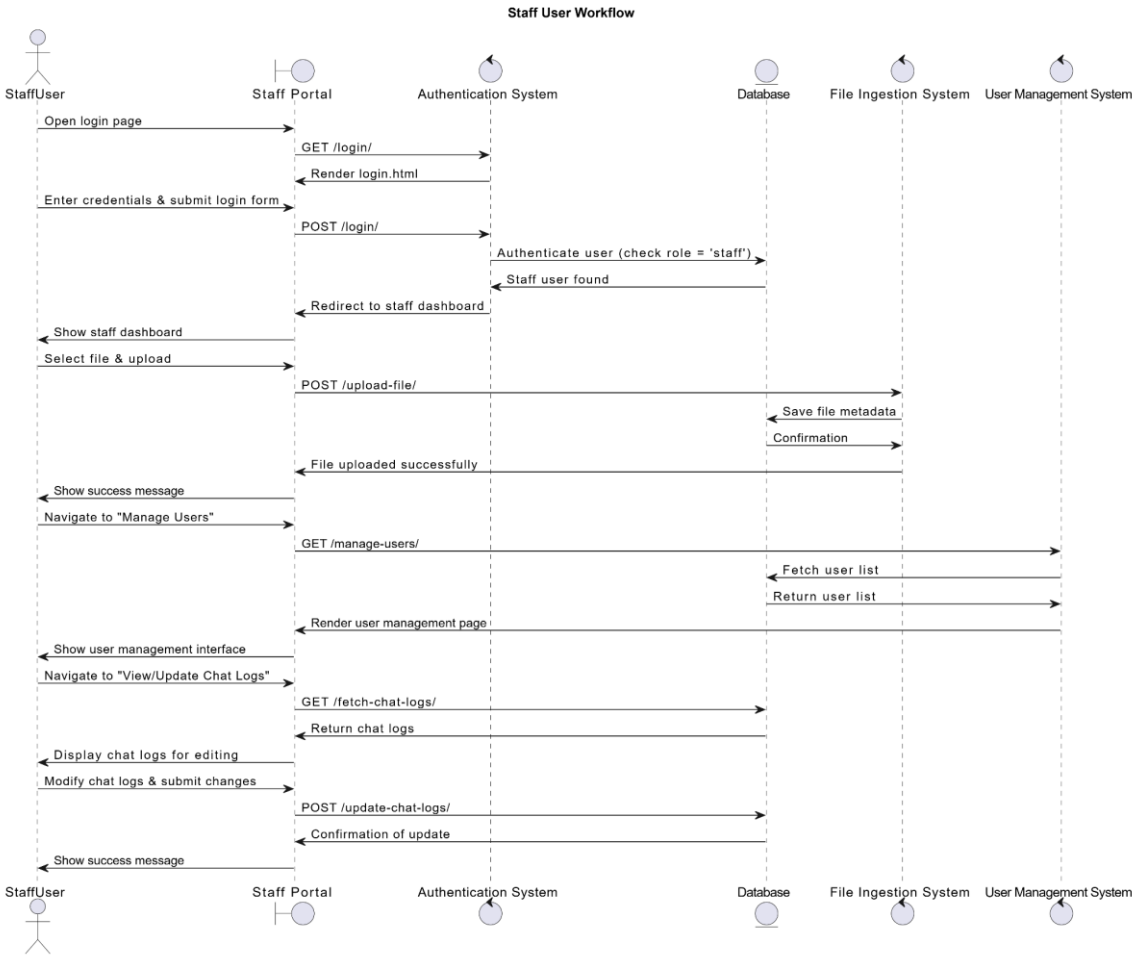


Figure 9: Dynamic modeling of Staff User Workflow using sequence diagram



Figure 10: Dynamic modeling of Super User Workflow using sequence diagram

3.1.3.3 Process Modeling

Process modeling is the graphical representation of business processes or workflows. A process model allows visualization of business processes so organizations can better understand their internal business procedures so that they can be managed and made more efficient [32].

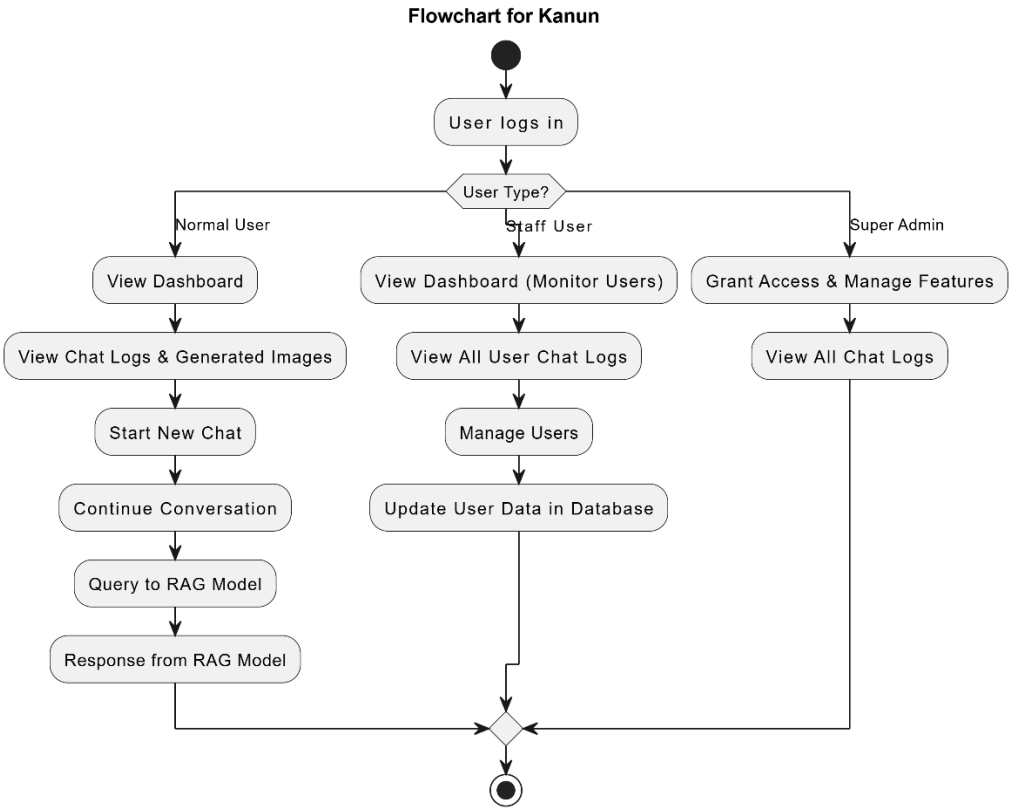


Figure 11: Process modeling of RAG-Based Application

CHAPTER 4: SYSTEM DESIGN

4.1 SYSTEM DESIGN

4.1.1 COMPONENT DIAGRAM

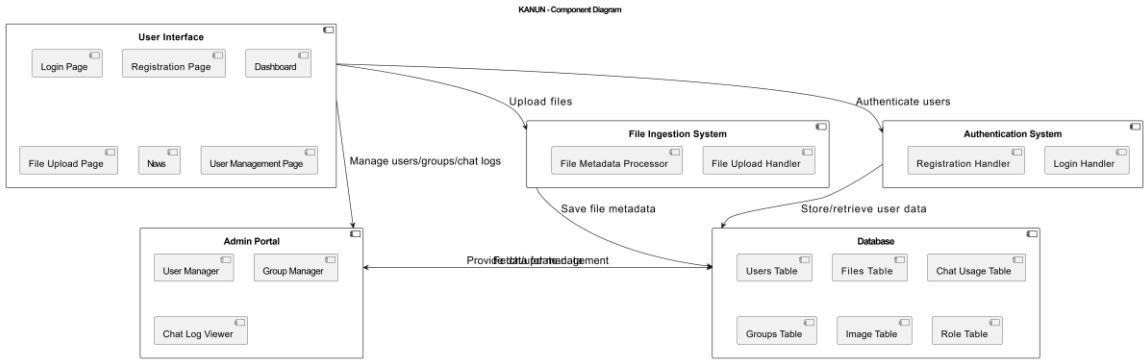


Figure 12: Object Modeling using Class Diagram of RAG- Based Application

4.1.2 DEPLOYMENT DIAGRAM

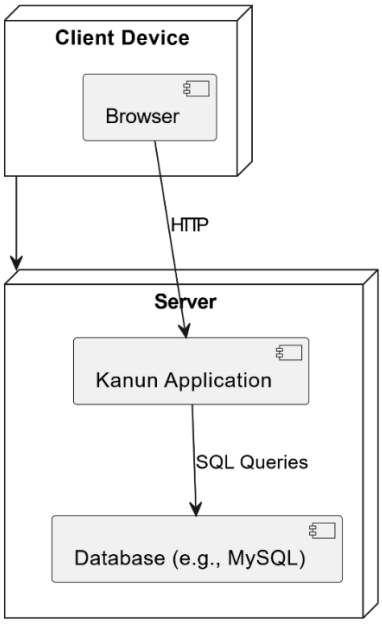


Figure 13: Deployment Diagram

4.2 ALGORITHM DETAILS

4.2.1 MAXIMUM MARGINAL RELEVANCE

Maximum Marginal Relevance (MMR) is one of the ranking methods for information retrieval and information filtering, which will be aimed at optimizing the ranking of documents provided to users by information retrieval systems. MMR was developed as a method for selecting documents relevant to the user's interests from among multiple documents. The method will rank documents based on both the relevance and diversity of each document, specifically emphasizing the selection of documents that are highly relevant but have low similarity to other options.[33]

$$\mathbf{MMR = \arg\max_{D_i \in D}[\lambda \cdot Sim(Q , D_i) - (1-\lambda) \cdot \max_{D_i \in S} Sim(D_i , D_j)]}$$

Where:

- Q: Query vector.
- Di : Candidate item vector.
- D: Set of all candidate items.
- S: Set of already selected items.
- sim(Q , Di): Similarity between the query and the candidate item.
- sim(Di , Dj): Similarity between two candidate items.
- λ: Trade-off parameter (lambda_mult in the code).

Algorithm of MMR:

1. Start with an empty set S.
2. Iteratively select the item Di
3. that maximizes the MMR score:
    a. High similarity to the query ($sim(Q , D_i)$) increases relevance.
    b. Low similarity to already selected items ($sim(D_i, D_j)$) increases diversity.
4. Add the selected item D_i to S.
5. Repeat until k items are selected.

4.2.2 COSINE SIMILARITY

Cosine similarity is a metric used to measure how similar the documents are irrespective of the size. Mathematically, it measures the cosine of the angle between two vectors

projected in a multi-dimensional space. The cosine similarity is advantageous because even if the two similar documents are far apart by the Euclidean distance there will be a chance where they may still be oriented closer together.[34]

$$similarity(A,B) = cos(\theta) = \frac{A \cdot B}{||A||\,||B||} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2 \sum_{i=1}^{n} B_i^2}}.$$

Where,

- A and B are the two vectors.
- A.B = Dot product of vectors A and B:

$$A.B = A_1B_1+A_2B_2+.....+A_nB_n$$

- ||A|| = Magnitude (Euclidean norm) of vector A:

$$||A|| = \sqrt{A_1^2+A_2^2+.....+A_n^2}$$

- ||B|| = Magnitude of vector B

$$||B|| = \sqrt{B_1^2+B_2^2+.....+B_n^2}$$

The value of cosine similarity ranges from -1 to 1:

- 1 indicates that the vectors are perfectly aligned and point in the same direction (maximum similarity).
- 0 suggests the vectors are orthogonal (perpendicular), meaning they share no similarity.
- -1 shows that the vectors are diametrically opposed (point in opposite directions).

4.2.3 RETRIEVAL AUGMENTED GENERATION

RAG(Retrieval Augmented Generation) is a Generative AI architecture that enhances Large Language Models (LLMs) by combining information retrieval with generative capabilities, using fresh and trusted data from authoritative internal knowledge bases and enterprise systems to produce more accurate, contextually relevant, and up-to-date responses.[35]

The RAG algorithm can be broken down into the following steps:

1. Input Query

- A user provides a query Q, such as "What are the fundamental rights mentioned in the Constitution of Nepal?"

2. Retrieve Relevant Documents

- The retrieval component queries a knowledge source (e.g., SQLite database or document corpus) to retrieve k most relevant documents D={d
- Each document di is represented as a vector using techniques like TF-IDF , word embeddings , or dense representations (e.g., from transformer-based models like BERT).

3. Score and Rank Documents

- The relevance of each document di to the query Q is scored using a similarity function S(Q,di). Common similarity functions include:
    - Cosine Similarity : Measures the cosine of the angle between two vectors.
    - Dot Product : Computes the dot product between the query vector and document vector.
    - BM25 : A probabilistic ranking function for information retrieval
- The top k documents with the highest scores are selected.

4. Generate Response

- The generation component uses a pre-trained language model to generate a response R conditioned on the retrieved documents D.

- The generative model computes the probability distribution over possible responses R given the query Q and the retrieved documents D:

$$P(R|Q,D) = \prod_{t=1}^{T} P(w_t | w_{<t}, Q, D)$$

where:

- wt is the t-th word in the generated response.
- w <t represents all words generated before wt.
- T is the total length of the generated response.

5. Output Response

- The final response R is returned to the user.

4.2.4 TRANSFORMER ARCHITECTURE

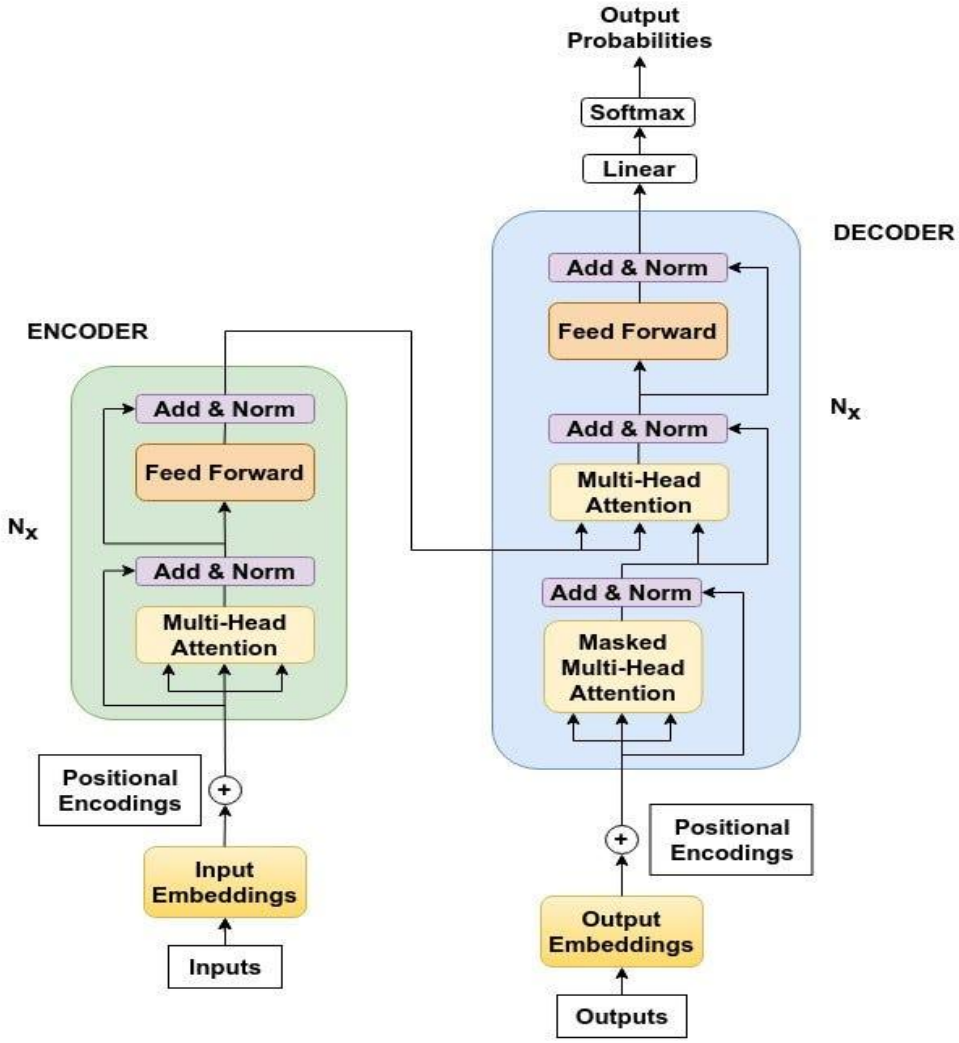


Figure 13: Transformer's Architecture

Transformers are a type of deep learning model that utilizes self-attention mechanisms to process and generate sequential data, effectively capturing long-range dependencies and contextual relationships. The purpose of this article is to explore the architecture and functionality of Transformers in deep learning. The Transformer model is based on an encoder-decoder structure, where both the encoder and decoder consist of multiple layers employing self-attention mechanisms and feed-forward neural networks. This design enables the model to process input data in parallel, making it highly efficient for tasks involving sequential data.

In the Transformer model, the encoder processes the input sequence and produces a set of continuous representations. These representations are then passed to the decoder, which generates the output sequence. The encoder and decoder work together to transform the input into the desired output, such as translating a sentence from one language to another or generating a response to a query. [36]

Working of Transformers

1. Input Representation

The first step in processing input data involves converting raw text into a format that the transformer model can understand. This involves tokenization and embedding.

Tokenization: The input text is split into smaller units called tokens, which can be words, sub words, or characters. Tokenization ensures that the text is broken down into manageable pieces.

- **Embedding:** Each token is then converted into a fixed-size vector using an embedding layer. This layer maps each token to a dense vector representation that captures its semantic meaning. Positional encodings are added to these embeddings to provide information about the token positions within the sequence.

2. Encoder Process in Transformers

- **Input Embedding:** The input sequence is tokenized and converted into embeddings, with positional encodings added.

- **Self-Attention Mechanism:** Each token in the input sequence attends to every other token to capture dependencies and contextual information. The self-attention scores are calculated and used to weigh the importance of different tokens.
- **Feed-Forward Network:** The output from the self-attention mechanism is passed through a position-wise feed-forward network, which consists of two linear transformations with a ReLU activation in between.
- **Layer Normalization and Residual Connections:** Layer normalization and residual connections are applied around the self-attention and feed-forward sub-layers to stabilize training and improve gradient flow.

Self-Attention and Feed-Forward Network Operations Within the Encoder

- **Self-Attention:** For each token, self-attention computes a weighted sum of the values (from the value matrix V), where the weights are determined by the dot product of the query (from the query matrix Q) and key (from the key matrix K) vectors.
- **Feed-Forward Network:** The output from the self-attention layer is transformed through a feed-forward network, which helps in learning complex representations.

3. Decoder Process

Input Embedding and Positional Encoding: The partially generated output sequence is tokenized and embedded, with positional encodings added.

- **Masked Self-Attention Mechanism:** The decoder uses masked self-attention to prevent attending to future tokens, ensuring that the model generates the sequence step-by-step.
- **Encoder-Decoder Attention Mechanism:** The decoder attends to the encoder's output, allowing it to focus on relevant parts of the input sequence.
- **Feed-Forward Network:** Similar to the encoder, the output from the attention mechanisms is passed through a position-wise feed-forward network.
- **Layer Normalization and Residual Connections:** Layer normalization and residual connections are applied to stabilize training and improve gradient flow.

Self-Attention, Encoder-Decoder Attention, and Feed-Forward Network Operations Within the Decoder

- **Masked Self-Attention:** Ensures that the decoder only attends to previous tokens in the sequence.
- **Encoder-Decoder Attention:** Allows the decoder to attend to the encoder's output, integrating information from the input sequence.
- **Feed-Forward Network:** Processes the combined output of the attention mechanisms to generate the final representation.

4. Training and Inference

Transformers are trained using supervised learning, where the model learns to predict the next token in a sequence given the previous tokens. The training involves the following steps:

- **Loss Function:** The most common loss function for training transformers is the cross-entropy loss, which measures the difference between the predicted and actual token distributions.
- **Optimization:** The model parameters are optimized using gradient descent techniques, typically with the Adam optimizer. The gradients are computed using backpropagation, and the model parameters are updated to minimize the loss.
- **Inference:** During inference, the trained model generates the output sequence step-by-step. For each step, the model predicts the next token based on the previously generated tokens and the input sequence.
- **Generating Output:** The decoder generates tokens until a special end-of-sequence token is produced or a predefined maximum length is reached. Beam search or other decoding strategies can be used to improve the quality of the generated sequence.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 IMPLEMENTATION

A LangChain-based application was developed to perform Retrieval-Augmented Generation (RAG) operations on the Constitution of Nepal. The development of the system was carried out in three phases. The first phase involved the integration of LangChain for building the retrieval system and incorporating preprocessors for handling constitutional data. The second phase focused on implementing the backend API and connecting it to the database for efficient data retrieval and storage. The final phase was the development of the frontend, which included the creation of a user-friendly interface for seamless interaction with the system that allows users to query the Constitution and receive accurate responses.

5.1.1 TOOLS USED

The tools used to create this we based visual demonstration of the proposed system are as follows:

5.1.1.1 PROGRAMMING LANGUAGE

i. HTML and CSS

HTML and CSS are fundamental technologies for modern web development to enable the creation of visually appealing responsive websites. HTML and CSS is used to design and styling frontend of the "कानुन" RAG-based application. HTML is very simple to edit as there is no need for a special interface or platform to edit. It can be simply edited in any editor and also has added flexibility and interactivity. It is used for developing and styling the structure of webpage, contents and layout.[37]

ii. Bootstrap

Bootstrap is a free front-end CSS framework for faster and easier web development. It contains CSS and JavaScript based design templates. It contains HTML and CSS based design templates for typography, creating forms, buttons, tables, navigation, image carousels as well as optional JavaScript plugins. Bootstrap was developed by Mark Otto and Jacob Thornton at Twitter, and released as an open source product

in August 2011 on GitHub. Bootstrap 5 is the newest version of Bootstrap with new components, faster stylesheets and more responsiveness etc. It supports the latest and stable releases of all major browsers and platforms.[38]

iii. Python

Python is a high-level programming language that has gained more popularity due to its simplicity, readability and versatility. In 1991, It was released and created by Guido van Possum. Python played a central role in the development of the "कानुन" application for implementation of LangChain Framework for performing Retrieval-Augmented Generation (RAG) operations on the Constitution of Nepal Python's extensive libraries and frameworks ensure that developers can efficiently build, test and deploy such applications making it more powerful tools for both beginners and experts in the field of technology. The connection to the database was managed using Django's ORM which facilitated smooth data retrieval and interaction with the backend.[39]

iv. Django Framework

Django is a high-level Python web framework that enables rapid development of secure and maintainable websites that interact with the database and provide the information to the user. It was initially developed between 2003 and 2005 by a web team who are responsible for creating and maintaining newspaper websites. Django has the ability to handle database management, user authentication, API integration and security makes it an ideal choice for the project "कानुन". User-friendly interface is created due to a powerful feature of Django which allows users to query the constitution and receive accurate, contextually relevant responses. Django integration with LangChain and RAG ensures that the application can effectively process and generate answers based in Nepal making it a valuable tool for legal research and education.[40]

iv. SQLite3 Database

SQLite is a lightweight, file-based relational database management system (RDBMS) that does not require a separate server process. SQLite Database was used to store constitutional data and user's data. It was chosen for its simplicity and ability to handle rational data efficiently in the embedded format. It serves as a

system of database management for storing and loading the Nepal Constitution. Being lightweight without a server makes it an ideal choice for the project "कानुन". The smooth integration of SQLite with Django's ORM ensures that the application can smoothly process database operations making it a valuable tool for legal research and education. SQLite provides a simple but executive solution for managing the Constitution of Nepal data in "कानुन" RAG-based application allowing effective search and processing of legal information using LangChain and RAG techniques.[41]

v. LangChain

LangChain is an open-source framework for the development of applications using Large Language Models (LLMs).Ir provides a standard interface for chains, numerous integrations with other tools and end-to-end chains for common application. LangChain allows developers to build application that combines LLMs such as GPT-4 with external sources of computation and data.In October 2022, LangChain was launched by Harrison Chase. LangChain was integrated into the system for performing Retrieval-Augmented Generation (RAG) operations on the Constitution of Nepal. Applications made with LangChain provide great utility for a variety of user cases from straightforward question-answering and text generation tasks to more complex solutions that use an LLM as a "reasoning engine".[42]

vi. Visual Studio Code

Visual Studio Code is a free, lightweight but powerful source code editor that runs on your desktop and on the web and is available for Windows, macOS, Linux and Raspberry Pi OS. It comes with built-in support for JavaScript, TypeScript, and Node.js and has a rich ecosystem of extensions for other programming languages. Aside from the whole idea of being lightweight and starting quickly, Visual Studio Code has IntelliSense code completion for variables, methods, and imported modules; graphical debugging; linting, multi-cursor editing, parameter hints, and other powerful editing features; snazzy code navigation and refactoring; and built-in source code control including Git support. Much of this was adapted from Visual Studio technology.

Visual Studio Code was extensively used in our project. It was the main code editor used in our project. It was used to write and manage the Python Code for the development of backend using Django Framework, LangChain integration for RAG operations and SQLite3 database operation. It was also used to write Python Code for developing user interfaces. The LangChain library with Django framework was integrated with VS Code to write and test directly in the code editor. VS Code was also for debugging our application and refining the overall system performance.[43]

vii. Git and GitHub

Git is the most commonly used version control system, Git tracks the changes you make to files, so you have a record of what has been done and you can revert to specific versions should you ever need to. Git also makes collaboration easier, allowing changes by multiple people to all be merged into one source. Your files and their history are stored on your computer.[44]

GitHub is a web-based interface that uses Git, the open source version control software that enables multiple developers to collaborate on the project at the same time. GitHub provides a workflow for tracking the project in modifications, merging updates and maintaining version history which helps to decrease production time. [45]

Git was used to manage code changes in our application. It was used to track changes to our code and collaborate with team members. GitHub was used to create pull requests and review code changes between team members.

viii. Microsoft Word and Excel

MS Word is a processing software which is used for writing letters, essays, notes etc. MS Excel is a spreadsheet software where a huge amount of data or information can be saved in a systematic tabular manner in alphabetical and numerical values.[46]

Microsoft Word was utilized throughout the entire project to document the needs, procedures and workflow. Microsoft Excel was also used to keep tracking of the project's deadline.

xi. Google Drive

Google Drive was extensively used in our project. It was used to collaborate on documents, spreadsheets and presentations. It was used for sharing the files and folders between the team members. With its cloud-based infrastructure, we were able to securely access and collaborate on our files from any location with an internet connection which was particularly advantageous for team members who worked remotely or were frequently on the move.

x. Draw.io

Draw.io was a cloud based platform tool in our project because it provides us a versatile platform for creating diagrams, flowcharts, and other visual aids. Its cloud-based platform enabled us to work on diagrams in real time and easily share them with other team members, making it an invaluable tool for communication and project planning.

5.1.2 IMPLEMENTATION DETAILS OF MODULES

5.1.2.1 Code Snippet for User management

```
class CustomUserManager(BaseUserManager):
    def create_user(self, username, email, password=None, **extra_fields):
        if not username:
            raise ValueError("The Username field must be set")
        if not email:
            raise ValueError("The Email field must be set")

        email = self.normalize_email(email)
        user = self.model(username=username, email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
```

```
        return user

    def create_superuser(self, username, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)

        if extra_fields.get('is_staff') is not True:
            raise ValueError("Superuser must have is_staff=True.")
        if extra_fields.get('is_superuser') is not True:
            raise ValueError("Superuser must have is_superuser=True.")

        return self.create_user(username, email, password, **extra_fields)
```

5.1.2.2 Code Snippet for Creating User

```
class CustomUser(AbstractBaseUser, PermissionsMixin):
    id = models.AutoField(primary_key=True)
    username = models.CharField(max_length=150, unique=True)
    email = models.EmailField(null=True, blank=True)
    password = models.CharField(max_length=128)
    is_staff = models.BooleanField(default=False)
    is_superuser = models.BooleanField(default=False)

    objects = CustomUserManager()

    USERNAME_FIELD = 'username'
    REQUIRED_FIELDS = ['email']

    def __str__(self):
        return self.username
```

5.1.2.3 Code Snippet for creating form

```
class RegisterForm(forms.ModelForm):
    password1 = forms.CharField(widget=forms.PasswordInput, label="Password")
```

```
    password2 = forms.CharField(widget=forms.PasswordInput, label="Confirm
Password")

    class Meta:
        model = CustomUser
        fields = ['username', 'email']  # Include email field here

    def clean_password2(self):
        password1 = self.cleaned_data.get('password1')
        password2 = self.cleaned_data.get('password2')
        if password1 and password2 and password1 != password2:
            raise forms.ValidationError("Passwords don't match.")
        return password2

    def save(self, commit=True):
        user = super().save(commit=False)
        user.set_password(self.cleaned_data["password1"])
        if commit:
            user.save()
        return user

class LoginForm(AuthenticationForm):
    username = forms.CharField(label="Username")
```

5.1.2.3 Code Snippet for login and registration verification

```
def register_view(request):
    if request.method == "POST":
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, "Registration successful!")
            if user.is_superuser:
                return redirect("admin:index")
```

```
            else:
                return redirect("login")
        else:
            messages.error(request, "Registration failed. Please check the form and try
again.")
    else:
        form = RegisterForm()
    return render(request, "register.html", {"form": form})


# Login view (using Django's built-in authentication view)
class CustomLoginView(LoginView):
    form_class = LoginForm
    template_name = "login.html"

    def get_success_url(self):
        if self.request.user.is_superuser:
            return '/admin/'
        return '/dashboard/'

    def get_redirect_url(self):
        return self.request.path
```

5.1.2.4 Code Snippet for Dashboard View:

```
def home(request):
    users =
CustomUser.objects.annotate(chat_usage_count=Count('chatusagelog')).all()
    chat_logs = ChatUsageLog.objects.filter(user=request.user)

    images = GeneratedImage.objects.filter(user=request.user).order_by('-
created_at')

    paginator = Paginator(chat_logs, 10)
```

```
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    labels = [user.username for user in users]
    data = [user.chat_usage_count for user in users]

    context = {
        'labels': json.dumps(labels),
        'data': json.dumps(data),
        'users': users,
        'chat_logs': page_obj,
        'images': images,
    }
            return render(request, 'dashboard/home.html', context)
```

4.1.2.5 Code snippet for converting PDF to JSON

```
def convert_to_json_with_subclause_indexing(text):
    json_data = {}

    part_pattern = re.compile(r"(Part-\d+)\s*(.*?)\n", re.DOTALL)
    article_pattern = re.compile(r"(\d+)\.\s*([A-Za-z\s]+):\s*(.*?)(?=\n\d+\.|$)",
re.DOTALL)
    clause_pattern = re.compile(r"\((\d+)\)\s*(.*?)(?=\s*\(\d+\)|$)", re.DOTALL)
    subclause_pattern = re.compile(r"\((\w+)\)\s*(.*?)(?=\s*\(\w+\)|$)",
re.DOTALL)

    parts = re.findall(part_pattern, text)
    for part_number, part_title in parts:
        part_key = f"Part {part_number.split('-')[1].strip()}"
        part_data = {"title": part_title.strip(), "Articles": {}}

        articles = re.findall(article_pattern, text)
        for article_number, article_title, article_text in articles:
```

```
        article_key = f"Article {article_number.strip()}"
        article_data = {"title": article_title.strip(), "Clauses": {}}

        clauses = re.findall(clause_pattern, article_text)
        if clauses:
            for clause_number, clause_text in clauses:
                clause_key = f"Clause {clause_number.strip()}"
                clause_data = {}

                if "Explanation:" in clause_text:
                    clause_main, explanation = clause_text.split("Explanation:", 1)
                    clause_data["Text"] = clause_main.strip()
                    clause_data["Explanation"] = explanation.strip()
                else:
                    clause_data["Text"] = clause_text.strip()

                subclause_data = {}
                subclause_matches = re.findall(subclause_pattern, clause_text)
                for idx, (_, subclause_text) in enumerate(subclause_matches, 1):
                    subclause_data[f"SubClause {idx}"] = subclause_text.strip()

                if subclause_data:
                    clause_data["Subclauses"] = subclause_data

                article_data["Clauses"][clause_key] = clause_data
        else:
            article_data["Clauses"]["Clause 1"] = {"Text": article_text.strip()}

        part_data["Articles"][article_key] = article_data

    json_data[part_key] = part_data

return json_data
```

4.1.2.6 Code Snippet for Prompt Template 1

```
custom_prompt = PromptTemplate.from_template("""
Use the following pieces of context to answer the question at the end. If you don't know the
answer, just say that you don't know. Write answer in two paragraphs.
Context:
{context}
Chat History:
{chat_history}
Question:
{question}
Answer:
""")
```

4.1.2.7 Code Snippet for Follow up Prompt Template

```
rephrase_follow_up_prompt = PromptTemplate.from_template("""\
        First of all analyze the Chat History and Follow up questions.
        If the follow up question is not in context just reply you don't know.
        And if the follow up question is context reply accordingly.
        Chat History:
        {chat_history}
        Follow Up Input: {input}
        Standalone Question:
        """)
```

4.1.2.8 Code Snippet for Chat

```
chat_history.extend([
            {"role": "human", "message": query},
            {"role": "ai", "message": response}
        ])
        conversation.chat_history = chat_history
        conversation.save()
```

```
            return render(request, "chat/chat.html", {"chat_history": chat_history,
"conversation": conversation})
        prompt_with_context = custom_prompt.format(
            context=formatted_context,
            chat_history=format_chat_history(chat_history),
            question=rephrased_query
        )
        try:
            llm_response = llm.invoke(prompt_with_context)
            response = llm_response.content.strip()
            token_usage = llm_response.response_metadata.get("token_usage",
{}).get("total_tokens", 0)
        except Exception as e:
            logger.error(f"Error in LLM prediction: {e}")
            response = "Sorry, I couldn't process your query at the moment."
            token_usage = 0
chat_history.extend([
            {"role": "human", "message": query},
            {"role": "ai", "message": response}
        ])
        conversation.chat_history = chat_history
        conversation.save()
```

4.1.2.8 Code Snippet for Image Generation

```
generate_image(request):
    """Generate an image using DALL·E and save it in the database."""
    image_url = None
    error_message = None

    if request.method == 'POST':
        query = request.POST.get("query", "").strip()
        if query:
            try:
```

```
            client = OpenAI()
            response = client.images.generate(
                model="dall-e-3",
                prompt=query,
                size="1024x1024",
                quality="standard",
                n=1,
            )
            image_url = response.data[0].url

            # Save the image details in the database
            GeneratedImage.objects.create(
                user=request.user,
                prompt=query,
                image_url=image_url
            )

        except Exception as e:
            error_message = f"Error generating image: {e}"
```

5.2 TESTING

Software testing is a process of examining the functionality of a software which is under test to improve its performance and capabilities. This process helps in verifying and validating the actual software with the expected requirements to ensure that the product is built in the right way and is defect free. It involves identifying product bugs and requirements that have not been met. It also involves execution of software/system components using manual or automated tools to evaluate one or more properties of interest. The purpose of software testing is to identify errors, gaps or missing requirements in contrast to actual requirements.

5.2.1 UNIT TESTING

Unit testing is a software testing technique in which individual units or components of an application are tested in isolation from the rest of the application to ensure that they function as intended. In unit testing, each unit is tested separately to verify that it behaves as expected and produces the correct output for a given input.

5.2.1.1 Test Case 1

Table 9: Test Case For User Registration

Test Case ID	Test Case Description	Test Data	Expected Outcome	Pass/Fail Criteria
TC_REG_001	To verify the user gets registered with valid details.	Username:Ram Email: ram@gmail.com, Password: ram@1111	Users successfully get registered.	
TC_REG_002	To verify the registrations whether it fails with multiple repetition of username.	Username:Raju Email: raju@gmail.com, Password:raju@123	Error message: A user with that username already exists.	
TC_REG_003	To verify registration failure when the username input field is empty.	Username:``, Email: sita@gmail.com Password: sita@122	Error message: A username is required	
TC_REG_004	To verify the strength of the password validation.	Username:hari, Email: hari@gmail.com, Password: 123	Error message: Password must be at least eight characters long and contain a alphabet, numbers and special character	

5.2.1.2 Test Case 2

Table 10: Test Case For User Login

Test Case ID	Test Case Description	Test Data	Expected Outcome	Pass/Fail Criteria
TC_LOG_001	Verify that a user can Login with valid credentials.	Username:Ram Email: ram@gmail.com, Password: ram@1111	Users successfully log in.	
TC_LOG_002	Verify Login failure with an incorrect password	Username:Raju Email: raju@gmail.com, Password:raju@123	Error Message: Incorrect password.	
TC_LOG_003	Verify Login failure when the username does not exist.	Username:Unknown, Email: sita@gmail.com Password: sita@122	Error message: Username doesn't exist.	
TC_LOG_004	Verify Login failure when password field is empty.	Username:hari, Email: hari@gmail.com, Password:	Error message: Password is required	
TC_LOG_005	Verify Login failure whether both username and password field are empty.	Username: Email: sita@gmail.com password:	Error Message: Username and Password are required	

5.2.1.3 Test Case 3

Table 11: Test Case For Dashboard

Test Case ID	Test Case Description	Test Data	Expected Outcome	Pass/Fail Criteria
TC_DASH_01	Verify the Dashboard loading successfully.	User Login with valid credentials	Dashboard loaded successfully with all UI elements visible.	Pass if the dashboard displays correctly
TC_DASH_02	Verify Functionality of View User button .	Click on view users button	Redirects to the user list page	Pass if the user list loads.

TC_DASH_03	Verify Manage Users button functionality	Click on the manage users button.	Redirects to the user management page	Pass if management options are accessible
TC_DASH_04	Verify Chat Log button functionality	Click on the chat log button.	Displays chat history details.	Pass if chat log loads properly
TC_DASH_05	Verify the menu links navigation .	Click on links (Dashboard, Ingest, My Conversation, etc).	Navigates correctly to the respective sections.	Pass if all links function correctly
TC_DASH_06	Verify functionality of Logout.	Click on the logout button.	User is logged out and redirected to the login page.	Pass if session is terminated

5.2.1.4 Test Case 4

Table 12: Test Case For Chat Module

Test Case ID	Test Case Description	Test Data	Expected Outcome	Pass/Fail Criteria
TC_CHAT_01	Verify that a user can start a new chat.	Click on start new chat and type a question related to Constitution of Nepal	Chat initializes and the system responds with relevant constitutional information.	Test passes if the chat is initiated and a response is generated
TC_CHAT_02	Verify that it answers the constitution related question or not.	Enter question about Constitution of Nepal	System should reject the query with an " I don't know how to respond " message.	Test passes if the system prevents irrelevant queries.
TC_CHAT_03	Verify chat history was saved.	Start the conversation and ask multiple questions.	The chat history should display previous queries and responses.	Test passes if the chat history section updated correctly
TC_CHAT_04	Verify the real-time response generation.	Ask the constitution	System should generate and response within	Test passes if response time is acceptable

		related question	a reasonable time	
TC_CHAT05	Verify responsiveness for mobile and desktop	Access chat modules from different devices.	Chat interface should adjust for screen size	Test passes if the UI adapts correctly
TC_CHAT_06	Verify logout clears chat history and saved into my conversation	Logout and login again	Previous chat history should be visible in my conversation.	Test passes if logout saved chat history into my conversation

5.2.1.4 Test Case 4

Table 13: Test Case For Chat Module

Test Case ID	Test Case Description	Test Data	Expected Outcome	Pass/Fail Criteria
TC_IMG_01	Verify that the image generator accepts a valid prompt input.	Prompt: " A sunset over the mountains "	Successfully generates an image by the system	Test passes if the image is generated successfully.
TC_IMG_02	Verify the image generation fails with an empty prompt.	Prompt: " "	Error message: " Please fill out this field "	Test passed if proper error message is displayed
TC_IMG_03	Verify handling of special characters in the prompt	Prompt: "@#$%^&*()_-+!"	System processes or shows an error	Test passes if system does not crash and appropriate response.
TC_IMG_04	Verify that the generated image is displayed correctly.	Prompt: " A waterfall in a jungle"	Image appears with no visual glitches.	Test passes if the image is clear and properly displayed.
TC_IMG_05	Verify whether image saved into Gallery	Enter image generation prompt	Previously generated images should be visible in the Gallery.	Test passes if the image is saved into the gallery.

5.2.2 SYSTEM TESTING

System testing is a software testing technique used to evaluate the behavior of an entire software system or application, typically by testing the system as a whole rather than individual components or modules. The goal of system testing is to ensure that the software system works as intended, meets the specified requirements, and is free from defects and bugs that could negatively affect its functionality or usability

Table 14: System Testing

Test Case	**Test Scenario**	**Steps to Execute**	**Expected Result**	**Actual Result**	**Status**
Login	Verify that user can login successfully	1.Go to the Login 2. Fill out the form with valid details 3.Display dashboard to user	The user should be logged in and redirected to their dashboard.	Successfully login to the user dashboard	Pass
Registration	Verify that user can register successfully	1.Go to the /sign up or /register page. 2.Fill out the form with valid details. 3.Submit the form	The users should be registered and see a success message.	Users successfully register the application.	Pass
User Management	Verify that staff can access the details	1.Login with staff valid details. 2.Go to the user management 3.Details of valid users.	The staff user should be accessing the user details with the edit, delete and add button.	Staff users successfully get access to user details .	Pass
Chat Module	Verify that user can start new chat	1.Click on the start new chat 2. Type related queries to	The user can initialize the chat and respond with relevant constitutional information.	User successfully get respond with relevant constitutional information	Pass

		Constitution of Nepal			

5.3 RESULT ANALYSIS

Result analysis is the process involved in software development which involves the evaluation of the outcomes of various stages in the software development life cycle to ensure that project meets the objective and targeted requirements.

5.3.1 PROPOSED MODEL

The proposed model for Kanun application is a secured and legal information retrieval system that works on Retrieval-Augmented Generation (RAG) concept. The application is designed to help the user to retrieve the most relevant and accurate legal information. The Constitution of Nepal has been preprocessed into chunks which are converted into vector embedding using a pre-trained language model of OpenAI. A generative model then synthesizes these retrieved sections into coherent, human-readable responses and contextual explanation.

The Django framework as the backend handles the API requests and responses where a responsive frontend allows users to interact with the system via a web browser. This application enhances accessibility to legal information which makes it easier for citizens to get through the Constitution of Nepal including those without legal expertise to understand human rights and obligations. In Nepal, the proposed of Kanun application could help to acknowledge the citizen right toward the Nation.

Additionally, the use of RAG technology could increase the transparency and people can access rules and regulations as they need. The Application provides a powerful tool for citizens, legal professionals and policymakers in Nepal. However, implementing Kanun application using RAG technology in Nepal would require overcoming several challenges, such as ensuring secure and reliable internet connectivity and building trust among citizens in the new application.Therefore, careful planning and collaboration among stakeholders, including government agencies, civil society would be essential to successfully implementing the proposed Kanun application in Nepal. Kanun had the potential to

revolutionize how people interact with the Constitution of Nepal allowing them to recognize their rights and fulfill their responsibilities with full confidence against any crime.

5.3.2 ADVANTAGES OF PROPOSED MODEL

The proposed Kanun query application using RAG technology offers several advantages over the traditional learning systems. It ensures that responses are precise and accurate in the original text. A user-friendly interface designed by Django makes it easier for the people and the citizens of Nepal to submit queries and receive detailed information about the Constitution of Nepal. Citizens can learn their legal rights and values from anywhere in the world, making the process more accessible and convenient. The application can reduce the costs associated with the traditional learning systems. The need for books, legal advisors and libraries is eliminated, reducing the overall cost of the traditional learning system.

5.3.3 DISADVANTAGES OF PROPOSED MODEL

As the application is based on RAG, it requires a large database of pre-processed data, which can be an extensive resource commitment. Similarly, in order to store the embedded data of the documents requires complex setup of vector database and interactions are complex that can cause additional latency in the system. It requires specialized expertise for development and maintenance. Furthermore, the accuracy of responses hinges on the quality and completeness of the underlying data. If the training dataset is flawed or incomplete, the AI may generate inaccurate or misleading information, potentially leading to user confusion and misinterpretations of constitutional articles. This reliance on data quality also highlights the critical importance of robust data governance and curation processes. Finally, the handling of sensitive legal queries raises serious concerns about data privacy and security, demanding stringent safeguards to protect confidential information and prevent misuse. Addressing these challenges is crucial for the responsible and effective deployment of AI-powered legal tools.

5.3.4 RELEVANT SCREENSHOT

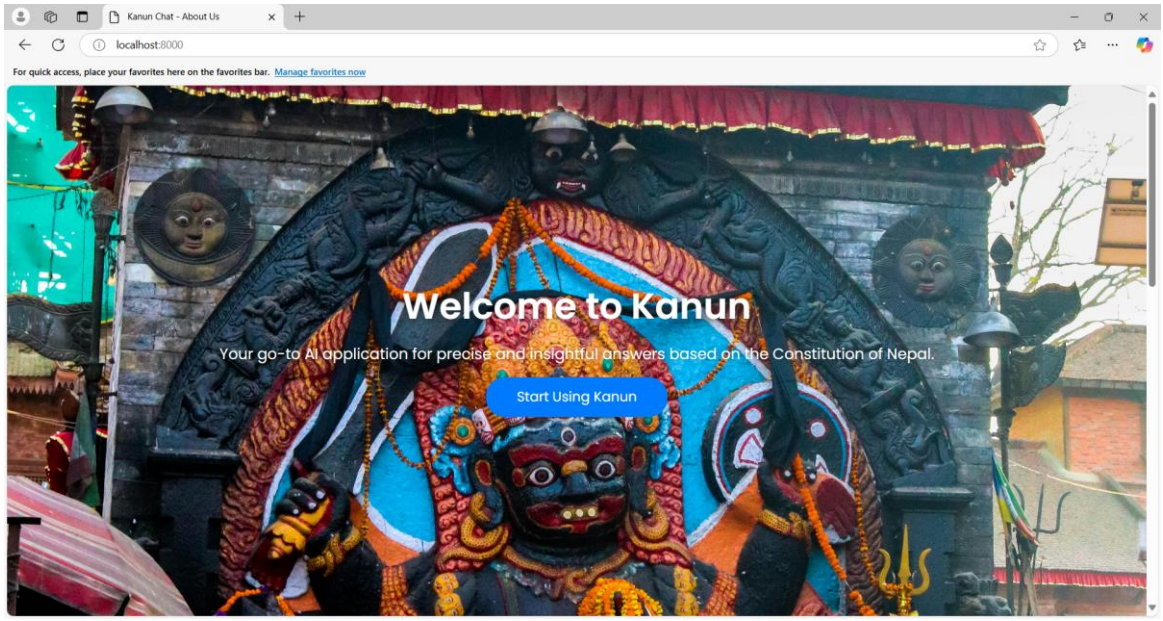


Figure 14: Cover Page

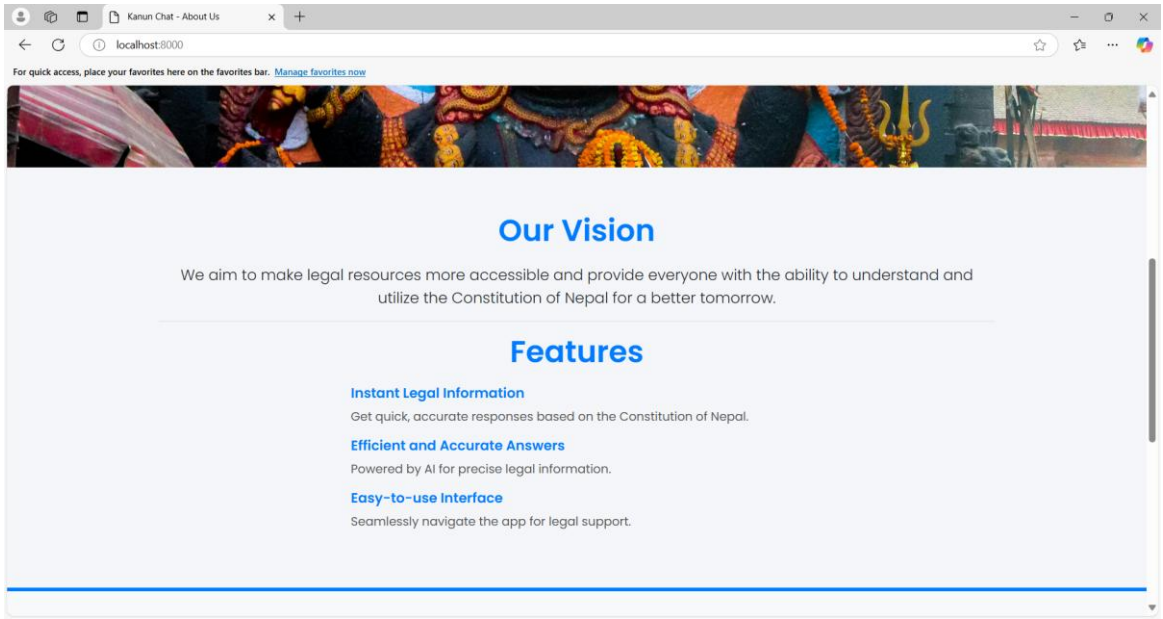


Figure 15: Cover Page - Introduction

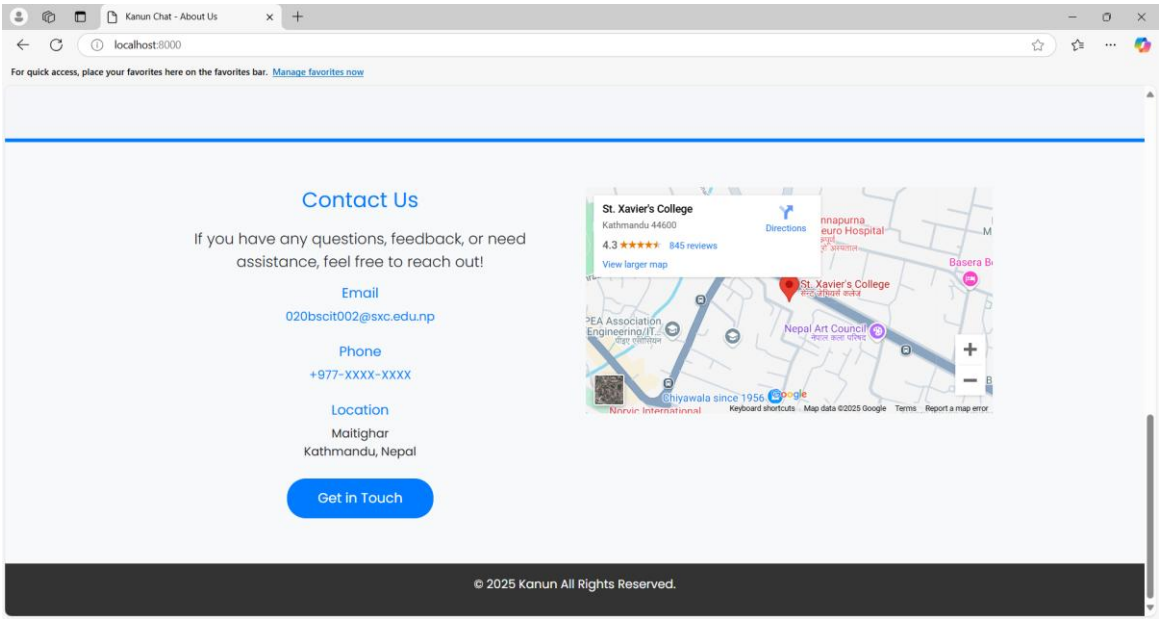


Figure 16: Cover Page - Contact

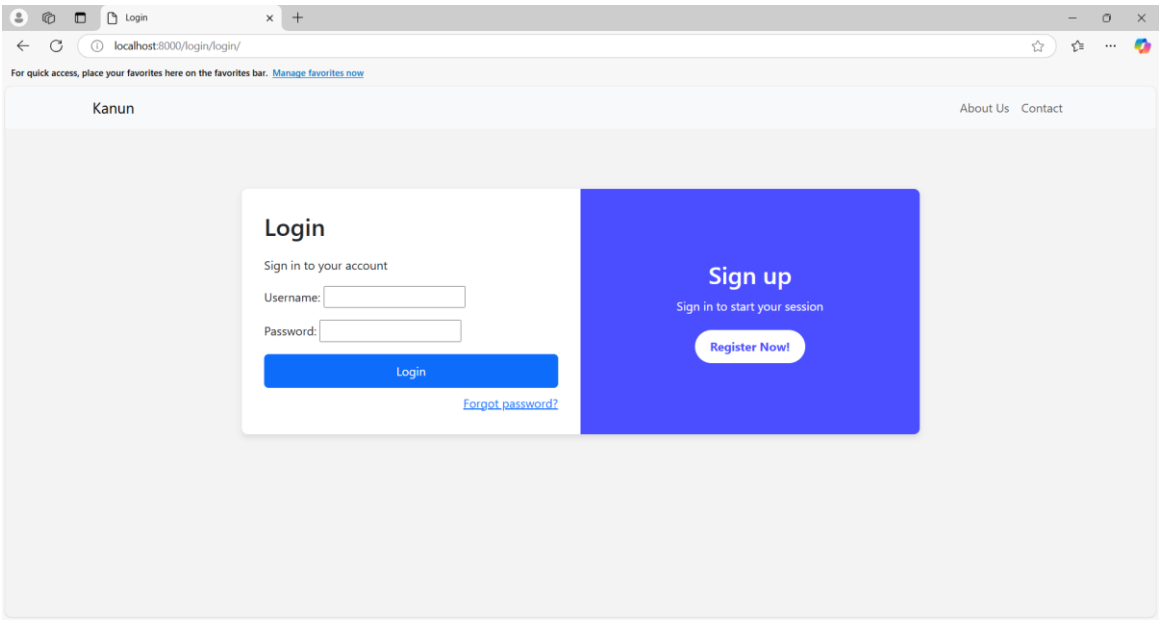


Figure 17: Login Page

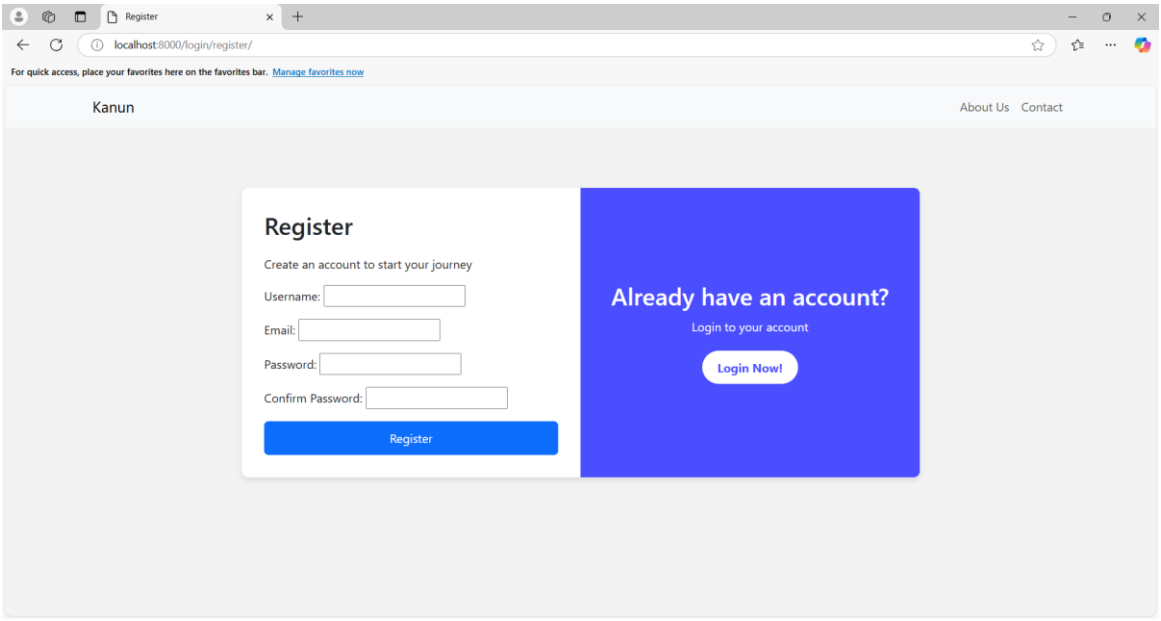


Figure 18: Registration Page

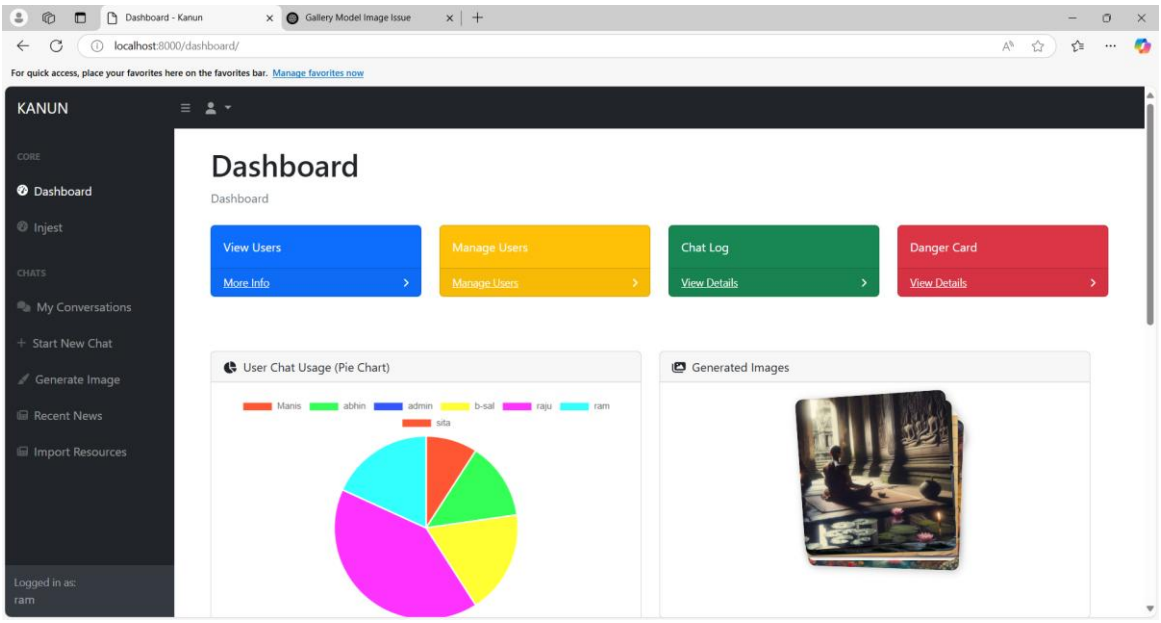


Figure 19: Dashboard Page

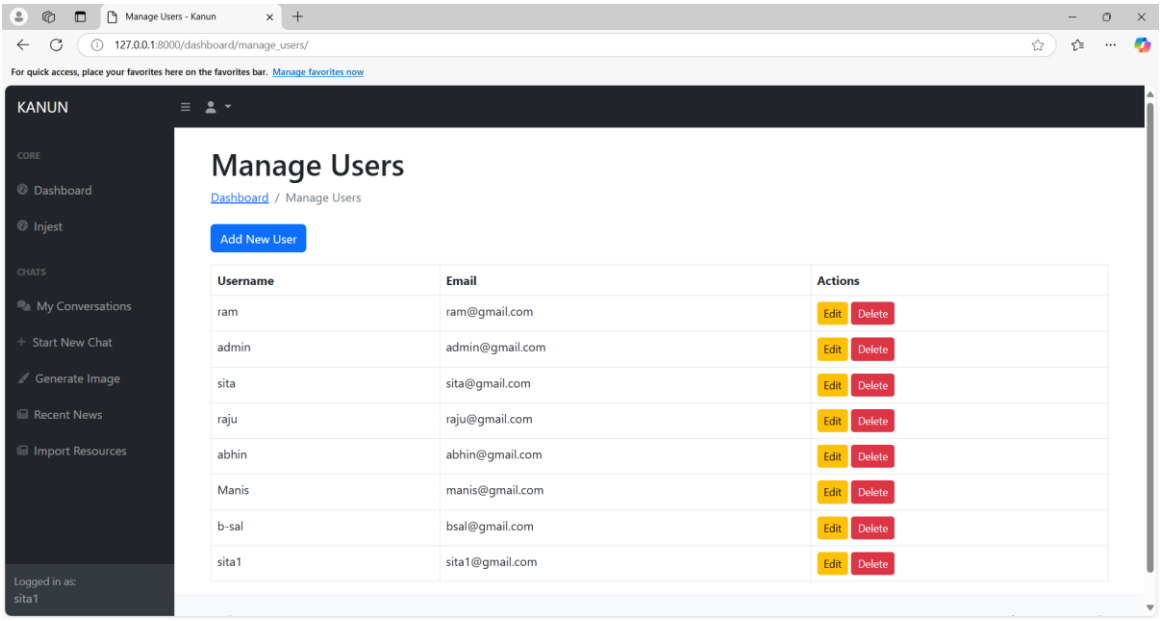


Figure 20: User Management

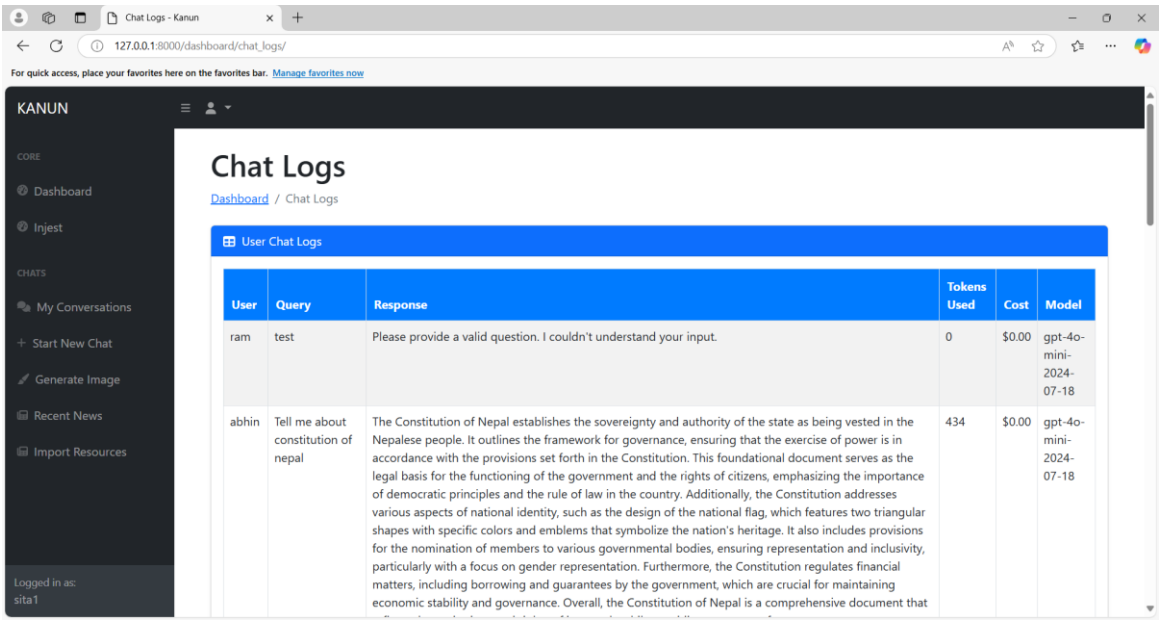


Figure 21: Chat Log

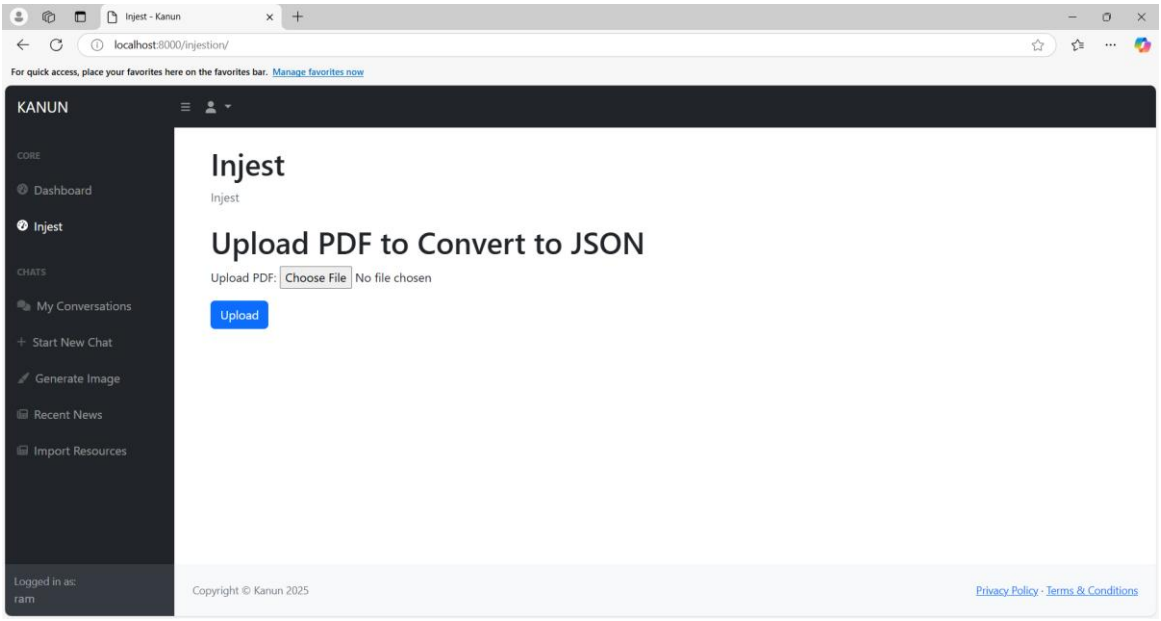


Figure 22: Injestion Page

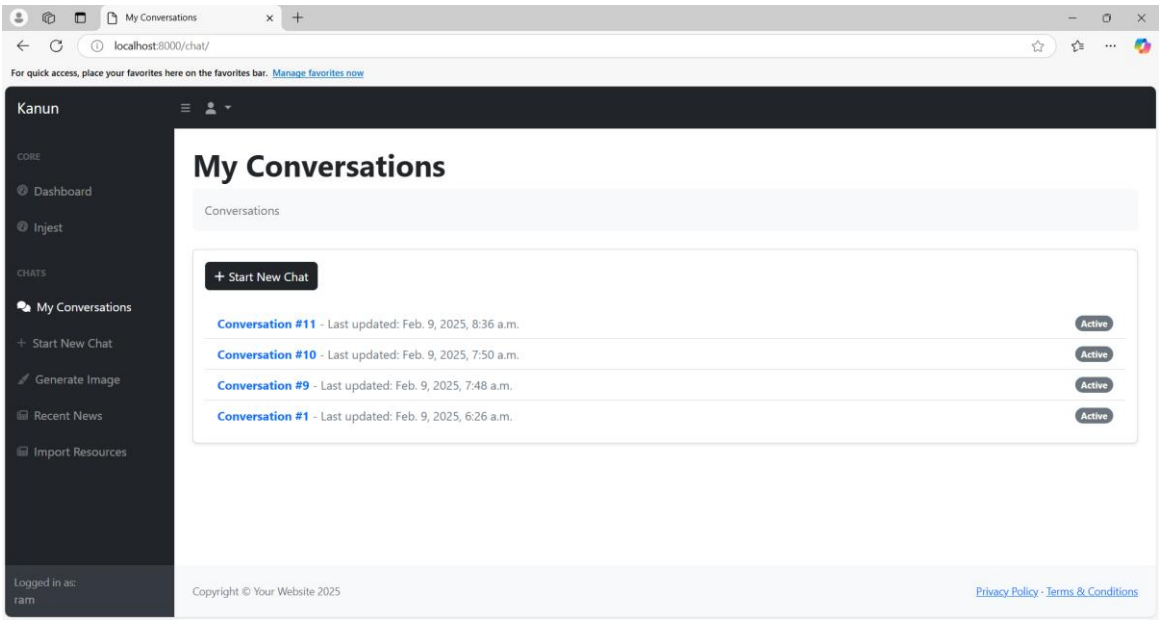


Figure 23: Conversation Page

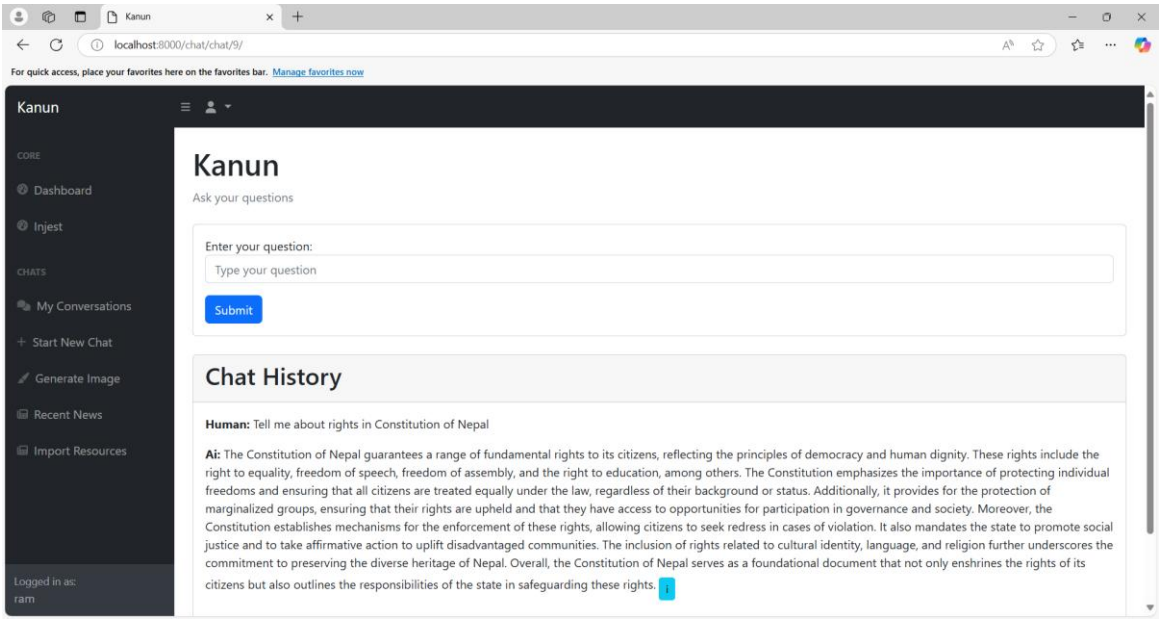


Figure 24: Chat Page

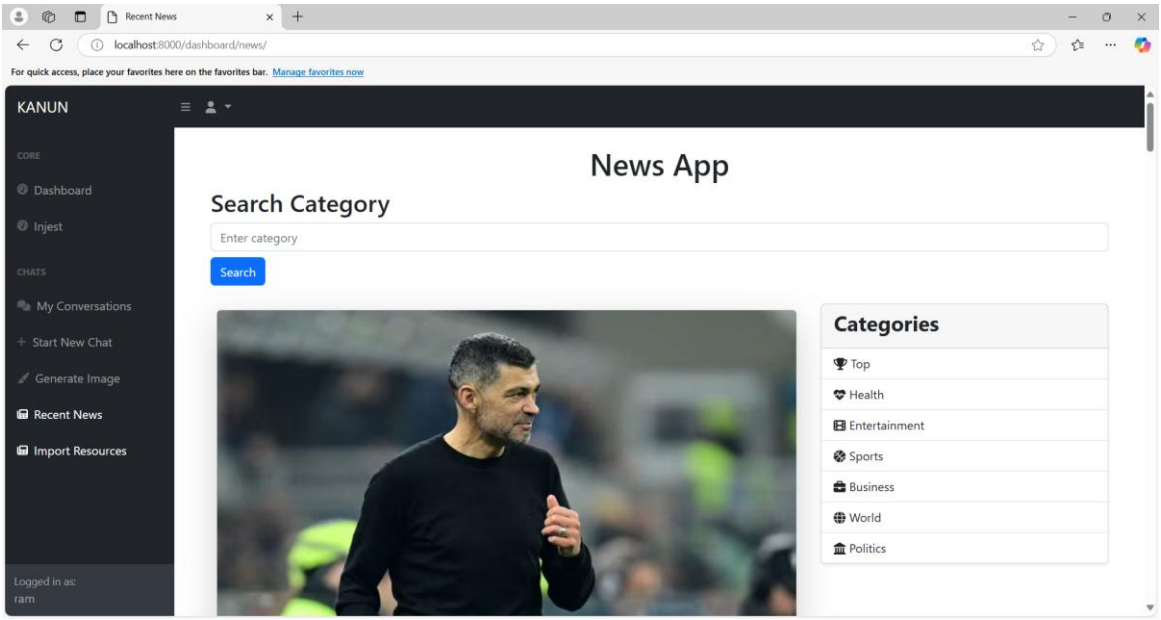


Figure 25: News Page

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION

6.1 CONCLUSION

The "कानुन" Kanun application, powered by Retrieval-Augmented Generation (RAG) technology, marks a notable step forward in democratizing access to Nepal's Constitution. It is designed centered around pre-processed constitutional text, which allows for the generation of clear and contextual responses to user inquiries. This approach aims to bridge the gap between complex legal language and the everyday citizen, fostering a deeper understanding of fundamental rights and obligations. The Django-based interface further simplifies interaction also this enhanced accessibility has the potential to cultivate greater transparency and citizen's participation and empowering individuals to navigate the legal framework with increased confidence. While the benefits are clear, including improved accuracy and cost-effectiveness compared to traditional legal resources, certain implementation hurdles must be addressed. These include the considerable effort required for data preparation and storage, the technical expertise needed for ongoing maintenance, and the paramount importance of data integrity for ensuring reliable outputs. Similarly, data management and security protocols are vital to safeguard sensitive information and build public trust. Overcoming these challenges is essential to fully realize the transformative potential of the Kanun application and its impact on legal literacy in Nepal. Despite these complexities, the Kanun application offers a promising avenue for reshaping the way citizens engage with the Constitution, ultimately fostering a more informed and empowered populace.

6.2 FUTURE RECOMMENDATION

The "कानुन" (Kanun) application, leveraging the power of Retrieval-Augmented Generation (RAG) technology, stands as a beacon of progress in democratizing access to Nepal's Constitution. Its core functionality, centered around pre-processed constitutional text and a sophisticated language model, promises to bridge the gap between complex legal jargon and the understanding of everyday citizens. This innovative approach aims to empower individuals with a clearer grasp of their fundamental rights and obligations, fostering a more informed and engaged citizenry. The user-friendly interface, built upon

the robust Django framework, further simplifies interaction, making the system accessible even to those without formal legal training. This enhanced accessibility has the potential to cultivate greater transparency and civic participation, enabling individuals to navigate the legal landscape with increased confidence.

However, the path to realizing the full potential of the Kanun application is not without its challenges. While the benefits are undeniable, including improved accuracy, enhanced convenience, and greater cost-effectiveness compared to traditional legal resources, several implementation hurdles must be carefully addressed. These include the considerable effort required for meticulous data preparation and storage, the technical expertise necessary for ongoing system maintenance, and, perhaps most importantly, the paramount importance of data integrity for ensuring the generation of reliable and accurate outputs. Moreover, the development and implementation of robust data management and security protocols are absolutely vital to safeguard sensitive information and build and maintain public trust in the system.

Looking ahead, several key recommendations emerge as crucial for the continued development and refinement of the Kanun application. Continuous improvement of the underlying dataset, including regular updates with amendments, related legislation, and pertinent case law, is essential. Exploring multilingual support to broaden accessibility to a wider population is also a key consideration. Furthermore, the integration of advanced retrieval techniques, such as Recursive Abstractive Processing for Tree Organized Retrieval, holds immense promise for enhancing the system's ability to efficiently navigate and summarize complex legal information. This approach could significantly improve the speed and accuracy of information retrieval, providing users with concise and relevant summaries of constitutional articles and related legal provisions.

Beyond technical enhancements, a strong emphasis must be placed on user experience and accessibility. The development of a mobile application, along with the provision of offline access to key information, is crucial for reaching users in remote areas with limited internet connectivity. A robust user feedback mechanism will be invaluable for gathering insights and driving future development efforts. Finally, fostering collaboration with legal professionals, government agencies, and local communities is essential for ensuring the accuracy, relevance, and overall effectiveness of the Kanun application. By embracing these recommendations and maintaining a commitment to ethical AI practices, the Kanun application can truly revolutionize the way citizens interact with the Constitution, ultimately fostering a more informed and empowered populace in Nepal.

REFERENCES

[1] Narayani Law Firm, "Constitution of Nepal 2072" [Online], Available: https://narayanilawfirm.org.np/constitution-of-nepal-2072/

[2] I. Sommerville, *Software Engineering*, 9th ed. Boston, MA, USA: Addison-Wesley, 2011, p. 32.

[3].K. D. Ashley, *Artificial Intelligence and Legal Analytics: New Tools for Law Practice in the Digital Age*. Cambridge, U.K.: Cambridge University Press, 2017.

[4] R. Susskind, *Online Courts and the Future of Justice*. Oxford, U.K.: Oxford University Press, 2019.

[5] Pinecone, "What is a Vector Database & How Does it Work? Use Cases + Examples" [Online], Available: https://www.pinecone.io/learn/vector-database

[6] P. Goyal, J. Chou, J. Liu, and N. Brown, "Efficient legal text retrieval using FAISS-based vector search," in *Proc. AI & Law Conf.*, 2021.

[7] M. Lewis, A. Piktus, and S. Riedel, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[8] Topsakal, O. and Akinci, T.C., 2023, July. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences* (Vol. 1, No. 1, pp. 1050-1056).

[9] Jeong, C., 2023. A study on the implementation of generative ai services using an enterprise data-based llm application architecture. arXiv preprint arXiv:2309.01105 e: https://arxiv.org/pdf/2309.01105

[10] M. E. Mamalis, E. Kalampokis, F. Fitsilis, G. Theodorakopoulos, and K. Tarabanis, "A Large Language Model Agent-Based Legal Assistant for Governance Applications," in *Electronic Government: 23rd IFIP WG 8.5 International Conference, EGOV 2024, Ghent-*

Leuven, Belgium, September 3–5, 2024, Proceedings, vol. 14841, Springer, 2024, pp. 286–301, 10.1007/978-3-031-70274-7_18.

[11] Filipovska, E., Mladenovska, A., Bajrami, M., Dobreva, J., Hillman, V., Lameski, P. and Zdravevski, E., 2024, September. Benchmarking OpenAI's APIs and other Large Language Models for Repeatable and Efficient Question Answering Across Multiple Documents. *In 2024 19th Conference on Computer Science and Intelligence Systems (FedCSIS)* (pp. 107-117), https://annals-csis.org/Volume_39/drp/pdf/3979.pdf

[12] Chia, H., Zhang, G.Z., Russo-Batterham, D., Shrestha, K.M., Quang, U.T., Wang, Z. and Paterson, J.M., *LEGAL (Leveraging Expert Guidance for AI in Law): A RAG-Based System for Legal Document Navigation*, https://alta2024.alta.asn.au/assets/papers/39.pdf

[13] Lin, D., 2024. *Revolutionizing retrieval-augmented generation with enhanced PDF structure recognition. arXiv preprint arXiv:2401.12599*, https://arxiv.org/pdf/2401.12599

[14] McGinnis, John O., and Russell G. Pearce., 2013. "*The great disruption: How machine intelligence will transform the role of lawyers in the delivery of legal services*." Fordham L. Rev., 82, p.3041., https://heinonline.org/HOL/LandingPage?handle=hein.journals/flr82&div=96&id=&page= [Accessed 12 December 2024].

[15] Azirar, S., Gabbar, H.A. and Regoui, C., 2024. "*IQLS: Framework for leveraging Metadata to enable Large Language Model based queries to complex, versatile Data.*" arXiv preprint arXiv:2405.15792., https://arxiv.org/pdf/2405.15792

[16] Nigam, S.K., Deroy, A., Maity, S. and Bhattacharya, A., 2024. "*Rethinking legal judgement prediction in a realistic scenario in the era of large language models.*" arXiv preprint arXiv:2410.10542., https://arxiv.org/pdf/2410.10542

[17] Wang, X., Zhang, X., Hoo, V., Shao, Z. and Zhang, X., 2024. LegalReasoner: A Multi-Stage Framework for Legal Judgment Prediction via Large Language Models and Knowledge Integration. IEEE Access., https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10750819

[18] Sil, R., Roy, A., Bhushan, B. and Mazumdar, A.K., 2019, October. Artificial intelligence and machine learning based legal application: the state-of-the-art and future research trends. *In 2019 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS) (pp. 57-62). IEEE.*, Available: https://www.researchgate.net/profile/Riya-Sil/publication/338938049_Artificial_Intelligence_and_Machine_Learning_based_Legal_Application_The_State-of-the-Art_and_Future_Research_Trends/links/5e896f1f4585150839c0d5fa/Artificial-Intelligence-and-Machine-Learning-based-Legal-Application-The-State-of-the-Art-and-Future-Research-Trends.pdf

[19] Cheong, I., Xia, K., Feng, K.K., Chen, Q.Z. and Zhang, A.X., 2024, June. "(A) I Am Not a Lawyer, But...: Engaging Legal Experts towards Responsible LLM Policies for Legal Advice." In *The 2024 ACM Conference on Fairness, Accountability, and Transparency* (pp. 2454-2469)., Available: https://dl.acm.org/doi/pdf/10.1145/3630106.3659048

[20] Harasta, J., Novotná, T. and Savelka, J., 2024. It cannot be right if it was written by AI: on lawyers' preferences of documents perceived as authored by an LLM vs a human. *Artificial Intelligence and Law*, pp.1-38., Available:https://arxiv.org/pdf/2407.06798

[21] Pipitone, N. and Alami, G.H., 2024. LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. *arXiv preprint arXiv:2408.10343.*, Available: https://arxiv.org/pdf/2408.10343v1

[22] Gupta, V., 2024. Leveraging open-source models for legal language modeling and analysis: a case study on the Indian constitution. *arXiv preprint arXiv:2404.06751.*, Available: https://arxiv.org/pdf/2404.06751

[23] *"Systems Analysis: Definition & Example,"* Study.com, 2017. [Online]. Available: https://study.com/academy/lesson/systems-analysis-definition-example.html. [Accessed: 2-Feb-2025].

[24] J. Bishop, *Systems Analysis and Design*. Boston, MA: Course Technology, 2011.

[25] *Merriam-Webster's Dictionary of English Usage*. Springfield, MA: Merriam-Webster, 1989.

[26] J. Bishop, *Systems Analysis and Design*. Boston, MA: Course Technology, 2011.

[27]Editor (2019) Functional and non-functional requirements: Specification and types, AltexSoft. AltexSoft. Available at: https://www.altexsoft.com/blog/business/functional-and-non-functional-requirements specification-and-types/

[28] Scaled Agile framework, "Nonfunctional Requirements," [Online]. Available: https://scaledagileframework.com/nonfunctional-requirements/.

[29] J. Bridges, "What Is a Feasibility Study? How to Conduct One for Your Project," [Online], https://www.projectmanager.com/training/how-to-conduct-a-feasibility-study.

[30] The basics of the Object Model (no date) Section. Available at: https://www.section.io/engineering-education/basics-of-the-object-model/

[31] Lucidchart, "UML Sequence Diagram Tutorial," [Online]. Available: https://www.lucidchart.com/pages/uml-sequence-diagram

[32] Vanner, C. (ed.) What is process modeling?What is process modeling? 6 essential questions answered. Available at: https://www.bizagi.com/en/blog/what-is-process-modeling

[33] Deus Ex Machina, "Overview of Maximum Marginal Relevance (MMR) and examples of algorithms and implementations" [Online], Available: https://deus-ex-machina-ism.com/?p=65245&lang=en

[34] Machine Learning, "Cosine Similarity – Understanding the math and how it works (with python codes)"[Online], Available:https://www.machinelearningplus.com/nlp/cosine-similarity/

[35] TypingMind Blog, "RAG for Beginners: The Complete Guide to Retrieval Augmented Generation" [Online], Available: https://blog.typingmind.com/rag-for-beginners/

[36] GeeksForGeeks, “Architecture and Working of Transformers in Deep Learning ” [Online]. Available: https://www.geeksforgeeks.org/architecture-and-working-of-transformers-in-deep-learning/

[37] Academia.edu, “Introduction to HTML and CSS” [Online], Available: https://www.academia.edu/15540765/Introduction_to_HTML_and_CSS

[38] W3School, “Bootstrap Get Started” [Online], Available: https://www.w3schools.com/bootstrap/bootstrap_get_started.asp

[39] freeCodeCamp, “The Python Handbook – Learn Python for Beginners” [Online], Available: https://www.freecodecamp.org/news/the-python-handbook/

[40] Mozilla Developer, “Django introduction-Learn web development” [Online], Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django/Introduction

[41] Medium, “Using SQLite as a Database Backend in Django Projects” [ONLINE], Available: https://medium.com/@codewithbushra/using-sqlite-as-a-database-backend-in-django-projects-code-with-bushra-d23e3100686e

[42] IBM, "What is LangChain ?"[ONLINE], Available:https://www.ibm.com/think/topics/langchain

[43] Heller, M. (2022) What is visual studio code? Microsoft's extensible code editor, InfoWorld. InfoWorld. Available at: https://www.infoworld.com/article/2335960/what-is-visual-studio-code-microsofts-extensible-code-editor.html

[44]What is Git and why should you use it? (2022) What is Git and Why Should You Use It? Free Intro to Git Guide. Available at: https://www.nobledesktop.com/learn/git/what-is-git

[45] An introduction to github (2020) Digitalgov. Available at: https://digital.gov/resources/an-introduction-github/

[46] Byju's Exam Prep (2021) *Difference between MS word and MS excel - key differences*, *BYJUS*. BYJU'S. Available at: https://byjus.com/free-ias-prep/difference-between-ms-word-and-ms-excel/